结构简式视听识别与 3D 展示系统

摘要

针对化学学习中手写结构简式与分子空间形态难以直观关联、传统展示方式交互性不足的问题，设计并实现了一套基于 AMD Zynq-7020 的有机物视听识别与分子三维展示系统。系统面向 22 类预设有机物，支持摄像识别、语音类别输入、名称与化学式显示及分子三维展示，通过 HDMI 和局域网网页提供交互入口，网页端支持模型拖拽旋转、缩放与多角度观察，建立纸面表达、化学名称与空间结构之间的直观联系。

系统采用 PS/PL 协同架构：PL 侧通过 HLS 模块完成灰度转换、滤波及背景校正，由统一手写 RTL 加速器完成三层卷积、量化、池化及全连接计算；PS 侧完成笔迹定位、归一化、尺寸适配、语音识别与 HDMI 端三维渲染。针对片上资源受限与第三层卷积耗时较长的问题，采用四输出通道并行、输入特征广播和局部权重缓存，结合尾部参数 DDR 按需加载、片上特征缓存及加载器与量化器共享，实现一次启动完成整网推理。训练与验证所用数据集均由团队自主采集制作，形成从数据构建、模型训练到量化部署与硬件验证的完整流程，便于追溯样本来源、分析误识别原因和持续补充数据；通过量化感知训练与逐层整数参考对拍，保证软硬件迁移的数值一致性。

实验结果表明，量化模型在团队自制的 910 张开发验证图像上的准确率为 90.77%。同板、同定点模型下，132 组配对测试中，PS 单线程优化 C 与 PL RTL 完整调用的平均耗时分别为 48.933 ms 和 26.680 ms，实现 1.83 倍加速，耗时降低 45.48%，全部 22 维整数输出均与参考一致。最终整机占用 128.5 个 BRAM、123 个 DSP，布局布线满足时序约束，并通过两项回放及摄像、语音、三维展示、网页交互和正常停止验证。系统无需云端推理，为资源受限平台上的化学教学辅助与交互可视化提供了可复现的实现方案。

关键词：**Zynq、软硬件协同、定点 CNN、语音交互、分子可视化**。

目　录

第一部分　作品概述.....3
1.1 功能与特性.....3
1.2 应用领域.....4
1.3 主要技术特点.....4
1.4 主要性能指标.....5
1.5 主要创新点.....5
1.6 设计流程.....6
第二部分　系统组成及功能说明.....7
2.1 整体介绍.....7
2.2 硬件系统介绍.....8
2.3 软件系统介绍.....17
2.4 软硬件划分依据与接口设计.....24
2.5 模型训练与定点部署原理.....25
第三部分　完成情况及性能参数.....25
3.1 整体介绍.....25
3.2 特性成果.....26
3.3 创新设计与工程贡献详解.....31
第四部分　总结.....33
4.1 可扩展之处.....33
4.2 心得体会.....33
第五部分　大模型协作记录.....35
5.1 协作方式与记录说明.....35
5.2 资源约束下的 PL 迁移范围与软硬件划分.....35
5.3 Pool1 特征缓存复用与两层卷积前端优化.....36
5.4 定点 CNN 迁移的数值一致性控制.....37
5.5 统一 RTL 的 DDR 模型缓存与安全取消.....38
5.6 Conv3 四输出通道并行与整网集成纠错.....39
5.7 稳定帧交接约束下的预览调度优化.....40
5.8 异步结果一致性与硬件资源安全退出.....40
5.9 将校验经验提炼为可复用技能.....42
5.10 复现交付中的版本与运行状态管理.....42
5.11 性能指标与技术表达的人工复核.....43
5.12 团队对 AI 协作的复盘与心得.....44
第六部分　技能包提炼过程.....45
6.1 技能来源与通用逻辑提炼.....45
6.2 Overlay 文件与 HWH 接口检查.....45
6.3 NPY 与 NPZ 整数输出严格比对.....46
6.4 验证证据与复用边界.....46
第七部分　复现说明.....47
7.1 打开工程并运行仿真.....47
7.2 生成 bit/HWH.....47
7.3 制作上板运行 ZIP.....48
7.4 上传、校验并解压.....49
7.5 启动作品.....50
7.6 检查摄像、语音和三维展示.....50
7.7 运行板端固定输入验证.....51
7.8 正常停止并下载结果.....51

第一部分　作品概述

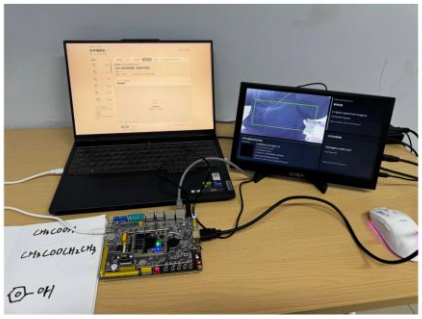
有机化学学习需要把纸面结构简式与分子的空间形态联系起来。课堂演示中，书写、查找物质信息和切换三维模型往往是分开的操作。本作品选择 22 类预设有机物作为教学对象，将手写摄像输入、化学名称语音输入和三维观察连接为同一交互流程，降低演示中的输入与切换负担。

已有工作分别提供化学图像识别和分子可视化能力：DECIMER.ai 研究从化学结构图像提取机器可读结构，3Dmol.js 提供浏览器分子展示与交互。本作品的工作重点是资源受限 Zynq 平台上的定点 CNN 实现、软硬件协同及视听展示集成；当前采用固定类别识别后调用预设模型，不将其表述为任意分子连接关系重建，也不与通用结构识别工具直接比较准确率。

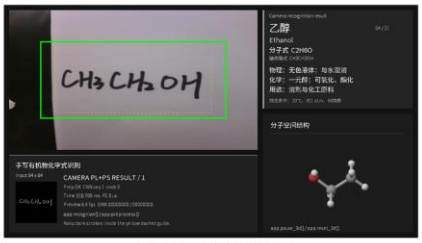
1.1 功能与特性

本作品面向化学学习，集成手写结构简式识别、语音输入与分子三维展示功能，支持 22 类预设有机物。摄像模式下，系统实时预览纸面图像，对识别区域内的手写结构简式进行处理，输出有机物名称、化学式及对应的预置分子三维模型；语音模式下，用户可通过语音选择有机物，查看其空间结构。

系统支持 HDMI 显示与局域网网页交互，可进行模式切换、结果清空和分子选择，并对空白、过暗、笔迹触边及低置信度输入进行提示。摄像采集、识别与界面显示由统一应用调度，使用结束后可正常停止并释放硬件缓冲资源，形成从输入、识别到可视化展示的完整交互流程。



(a) 系统硬件连接



(b) 手写结构简式识别



(c) 语音交互

图 1 系统实物与典型交互界面

1.2 应用领域

本作品主要面向有机化学教学、课后自主学习与科普展示，将纸面结构简式、化学名称和分子三维形态关联起来。

在课堂演示中，教师可书写系统支持的有机物结构简式，经摄像识别后展示其名称、分子式及三维模型，辅助讲解分子的组成与空间形态。在自主学习中，学习者可通过手写或语音输入调用对应分子，对照纸面表达观察三维结构。在科普展示中，系统提供摄像预览、语音交互和屏幕展示，便于体验不同输入方式与可视化结果之间的联系。

系统在本地完成识别与渲染，无需云端推理，支持 HDMI 显示和局域网网页操作，适合部署于教室、实验室及科技展示场景。当前识别范围为 22 类预设有机物。

1.3 主要技术特点

系统采用 AMD Zynq-7020 的 PS/PL 协同架构。PL 侧完成摄像采集、灰度转换、滤波、背景校正、CNN 推理及视频输出；PS 侧完成笔迹定位、归一化、尺寸适配、语音识别与分子三维渲染。

摄像识别采用三层定点 CNN，将卷积、量化、池化及全连接集成为统一手写 RTL 加速器，一次启动完成整网推理。通过 DDR 参数按需加载、片上特征缓存和计算资源共享，控制存储与计算开销。

系统提供 HDMI 与网页双端分子可视化。HDMI 端由 PS 生成球棍模型及旋转画面；网页上位机基于 3Dmol.js/WebGL，支持拖拽旋转、滚轮缩放、平移、自动旋转、速度调节及视角复位，便于从不同角度观察原子与化学键的空间关系。两端同步分子类别，网页视角独立调整，形成识别、选择与三维观察相结合的交互流程。

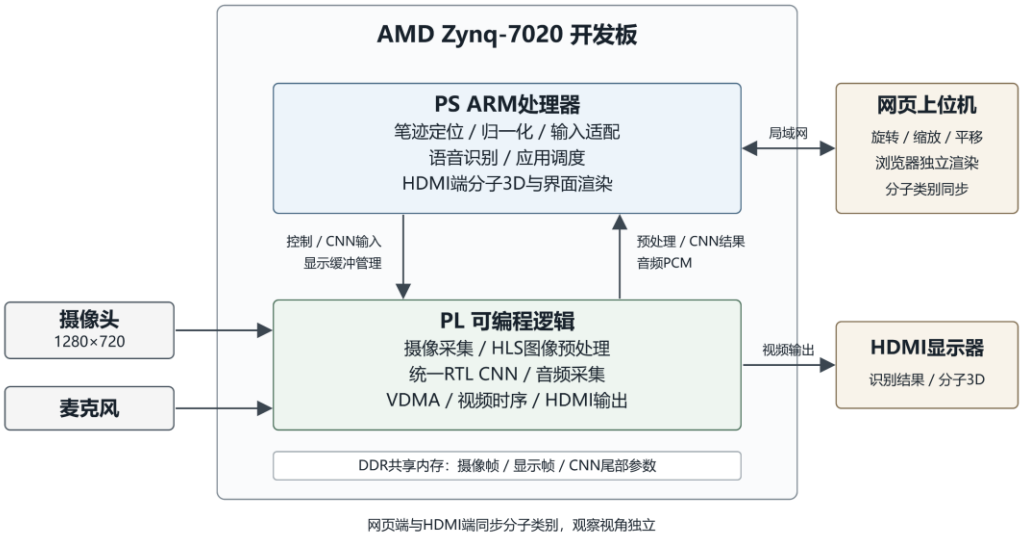


图 2 简化流程图

1.4 主要性能指标

指标	实测结果
识别类别	22 类预设有机物
PL 推理工作频率	100 MHz
PS 软件平均推理耗时	48.933 ms
PL RTL 平均推理耗时	**26.680 ms**
相对 PS 软件加速比	**1.83 倍，耗时降低 45.48%**
模型开发验证准确率	90.77%（826/910）
整机 LUT 占用	27921/53200（52.48%）
整机 BRAM 占用	128.5/140（91.79%）
整机 DSP 占用	123/220（55.91%）

注：同板、同定点模型，PS 采用单线程优化 C。RTL 计时含传输及控制，不含摄像、预处理和显示。准确率来自模型开发验证集（均为自制数据集）。

表 1 系统主要性能指标

1.5 主要创新点

（1）视听交互：摄像识别手写结构简式，语音识别化学名称，统一关联 22 类有机物信息与三维模型。

（2）双端 3D 展示：HDMI 呈现旋转球棍模型，网页支持拖拽旋转、缩放和平移，两端同步分子类别，网页视角独立调整。

（3）整网 RTL 加速：卷积至全连接一次启动，第三层四通道并行，较 PS 优化 C 加速 1.83 倍。

（4）自主数据与验证：训练、验证数据均自主制作，通过逐层整数对拍验证部署数值一致性。

1.6 设计流程

首先分析 22 类识别与交互需求，自制训练和验证数据集，完成模型训练、量化及整数参考构建；随后划分 PS/PL 任务，分模块实现 CNN 并逐层对拍，针对第三层瓶颈进行四通道并行优化；通过综合、布局布线检查资源与时序，最终开展实板功能验收及同板 PS/PL 配对测速，归档源码、构建脚本与验证证据。验证不一致时返回对应阶段修正。

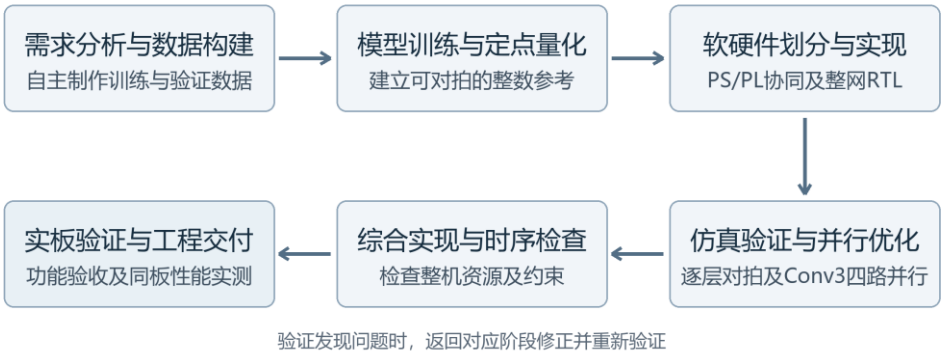


图 3 设计流程图

第二部分　系统组成及功能说明

2.1 整体介绍

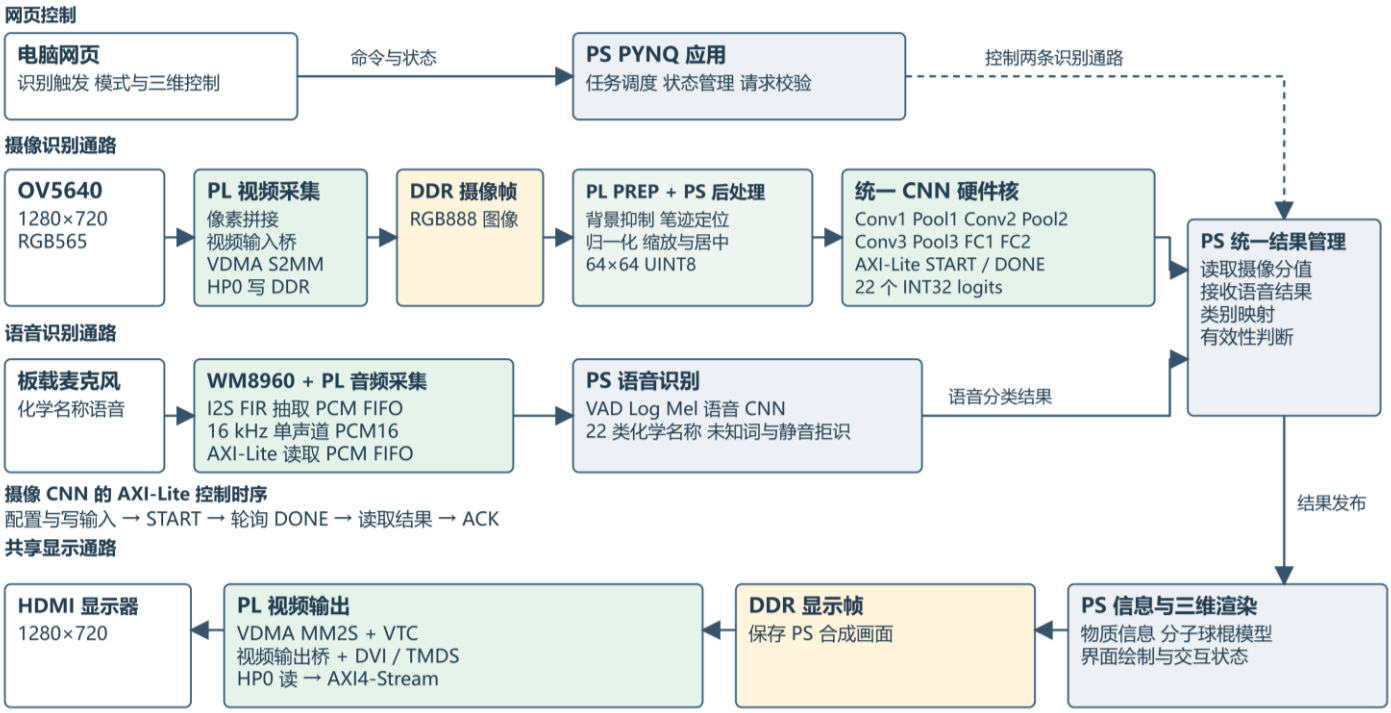


图 4 系统整体框图

系统以 Zynq 7020 为核心，采用 PYNQ Linux 与 PL 协同架构。摄像和语音两条识别通路汇入统一结果管理，再驱动物质信息与分子三维展示。

（1）摄像采集与图像预处理

OV5640 以 1280*720 RGB565 输出图像，PL 的 ov5640_capture_data_0 完成字节拼接与 RGB888 扩展，v_vid_in_axi4s_0 转为 AXI4 Stream，VDMA S2MM 经 HP0 将帧写入 DDR。PS 取得稳定帧后，调用 PREP 硬件完成背景与笔迹图处理，再由 PS C 完成定位、拒识、归一化、缩放与居中，得到 64 *64 UINT8 网络输入。

（2）统一 RTL 摄像识别

cnn_ps_if_0 当前封装完整的 cnn_rtl_unified 1.0。PS 写入输入窗口并执行一次 START，硬件依次完成 Conv1、Pool1、Conv2、Pool2、Conv3、Pool3、FC1 和 FC2，输出 22 个 INT32 logits。中间特征保存在片内 RAM，尾部参数通过只读 AXI 接口从 DDR 加载，PS 负责类别映射及结果有效性判断。

（3）语音采集与 PS 识别

板载麦克风通过 WM8960 转换为数字音频，audio_axi_0 完成 I2S 接收、FIR 抽取和 FIFO 缓冲，向 PS 提供 16 kHz 单声道 PCM16。PS C 执行 VAD、Log Mel 特征提取和专用语音 CNN，识别固定的 22 类化学名称，并处理未知词和静音拒识。该分支不经过摄像 RTL CNN。

（4）统一结果与显示输出

摄像和语音识别结果进入同一应用状态，关联物质名称、分子式、性质和预建分子模型。PS 生成界面及三维球棍模型，写入 DDR 显示缓冲；VDMA MM2S 读取显示帧，配合 v_tc_0、v_axi4s_vid_out_0 和 DVI_Transmitter_1 完成时序、视频转换与 TMDS 输出，经 HDMI 显示 1280*720 画面。

（5）PYNQ 控制与共享资源

电脑网页向 PS 应用提交操作，PYNQ 负责模式切换、任务调度、寄存器访问和缓冲管理。GPO 连接六个外设控制接口，HP0 承载视频帧读写，HP1 服务 PREP 图像处理与 CNN 参数读取。

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

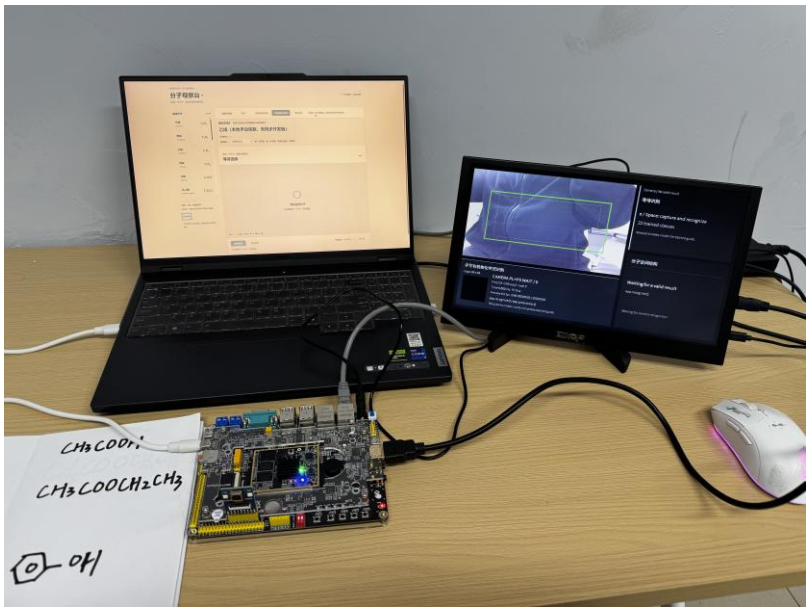


图 5 系统整体实物 摄像演示状态

整机由领航者 Zynq 7020 开发板、OV5640 摄像模块、板载音频采集器件、HDMI 显示器和电脑组成。开发板承担采集、识别与本地显示；电脑通过网页提供操作；外部显示器展示开发板输出的界面。纸面手写内容置于摄像视场内，语音输入则来自开发板麦克风。

实物	在系统中的职责	对应的设计部分
Zynq 核心板与底板	运行 PS 软件与 PL 电路，连接外设	processing_system7_0 及全部 PL 实例
OV5640 摄像模块	提供手写化学式图像	摄像采集 视频输入桥 VDMA S2MM
板载 MIC 与 WM8960	采集化学名称语音	audio_axi_0 与 audio_clock

实物	在系统中的职责	对应的设计部分
HDMI 显示器	显示预览 结果与三维模型	VDMA MM2S VTC 视频输出 TMDS
电脑与以太网连接	网页操作与状态查看	PS Linux 网络与应用服务

表 2 实物功能对照表

2.2.2 系统 Block Design

图 6 为工程的 Block Design 原图。PL 承担采集、计算与视频接口；PS 承担软件调度、语音识别和界面渲染。

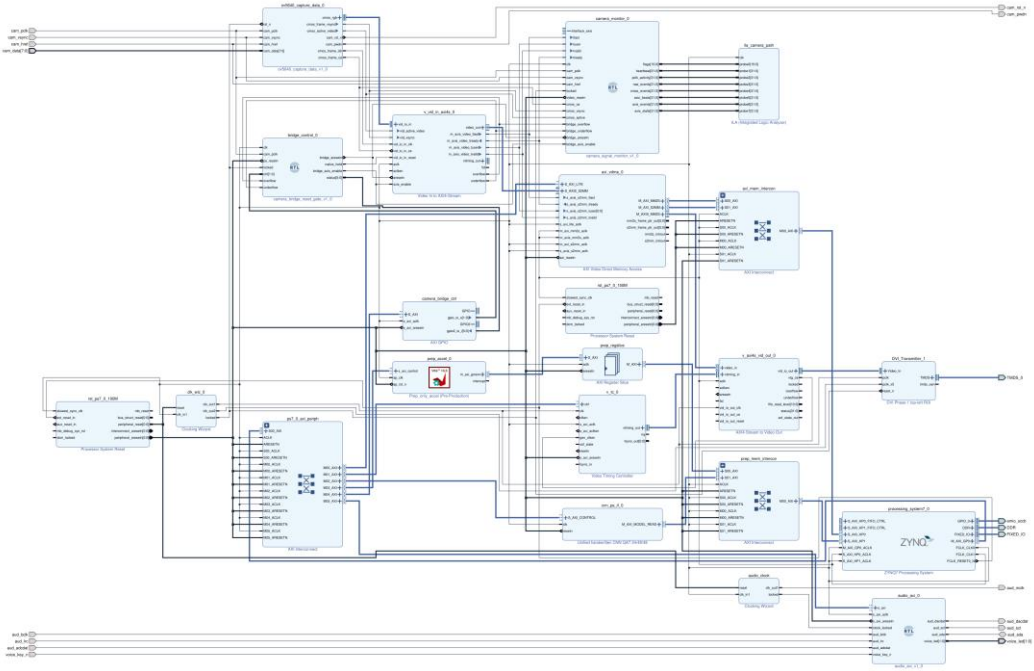


图 6 系统 Block Design 原图

2.2.3 电路各模块介绍

2.2.3.1 处理器系统与 AXI 互连

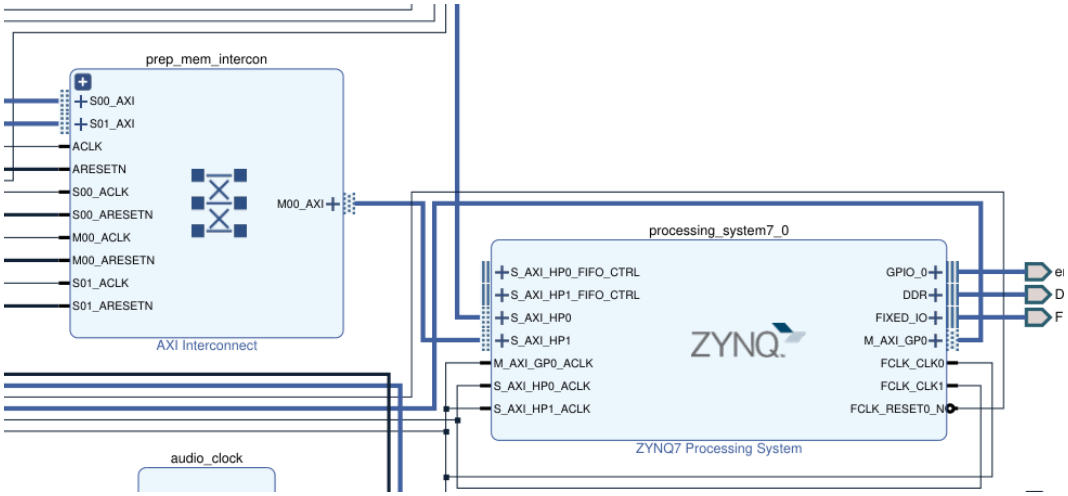


图 7 PS 与 HP1 内存互连局部

processing_system7_0 是系统控制与存储中心，包含 ARM Cortex A9 处理器系统及 DDR 控制接口。PS 运行 PYNQ Linux 软件，通过 AXI 控制 PL 外设，并通过 DDR 与 VDMA、预处理 IP 和 CNN 共享数据。

通路	原图中的连接	承担的任务
GP0 控制	M_AXI_GP0 → ps7_0_axi_periph → 六个控制接口	读写寄存器、配置帧地址、启动任务、写 CNN 输入及读结果
HP0 视频	axi_vdma_0 → axi_mem_intercon → S_AXI_HP0	S2MM 写摄像帧，MM2S 读显示帧
HP1 计算	PREP 与 CNN → prep_mem_intercon → S_AXI_HP1	PREP 读写图像；CNN 只读尾部模型参数

表 3 AXI 通路与功能

ps7_0_axi_periph 按地址将 PS 的访问送到目标外设。axi_mem_intercon 仲裁视频读写，prep_mem_intercon 仲裁 PREP 和 CNN 的内存请求，并完成所需的协议与位宽适配。prep_regslice 在 PREP 的 AXI 路径插入寄存器级以改善时序；它本身不执行图像处理，也不是跨时钟转换器。

GP0 中 PS 是访问发起方，因此端口为 M_AXI_GP0；HP0 和 HP1 接受 PL 发起的内存访问，因此 PS 端口为 S_AXI_HP0 和 S_AXI_HP1。AXI 读事

务仍会将 DDR 数据返回 PL，接口名称中的主从角色不等于数据只沿单一方向流动。

2.2.3.2 摄像采集与帧写入

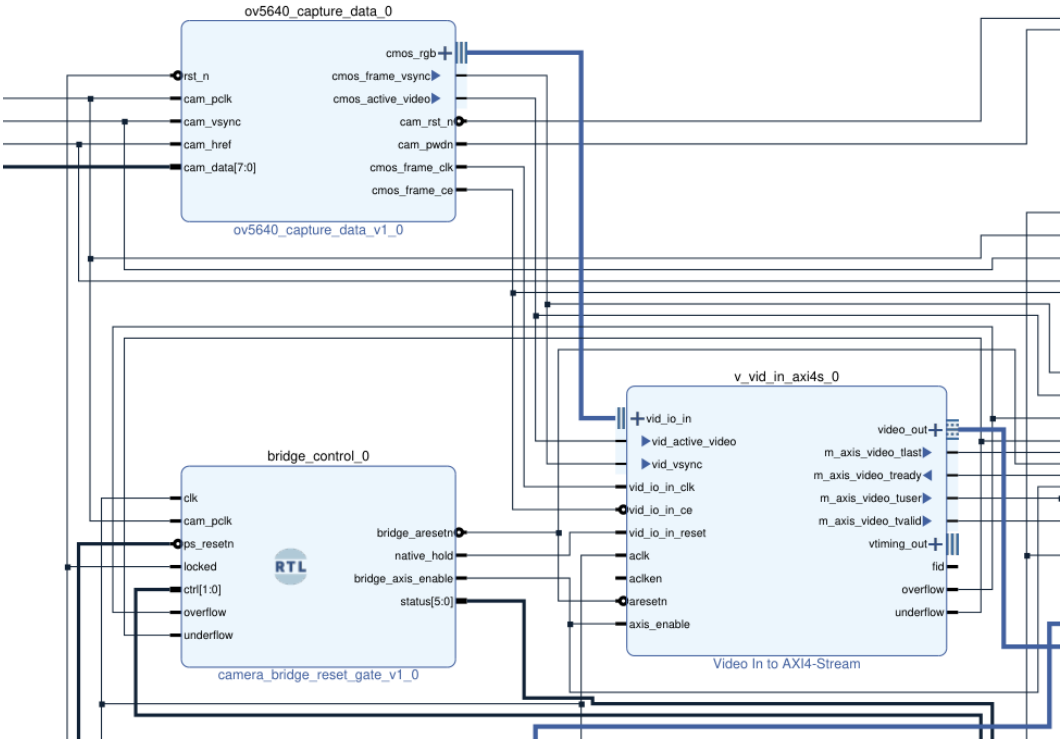


图 8 摄像采集 桥门控与视频输入局部

OV5640 摄像模块通过 cam_data 8 位数据线、cam_pclk、cam_href 和 cam_vsync 输出并行视频。当前传感器配置为 1280 × 720 RGB565。ov5640_capture_data_0 按像素时序拼接字节并扩展为 RGB888，输出原生视频数据及同步信息，同时提供摄像复位与掉电控制信号。

v_vid_in_axi4s_0 将原生视频转换为 AXI4 Stream。TVALID 表示上游数据有效，TREADY 表示下游可接收，两者同时有效时传输一个数据拍；TUSER 标记帧起始，TLAST 标记行结束。该模块衔接摄像像素域与 AXI 视频通路，输出接到 axi_vdma_0 的 S_AXIS_S2MM。

VDMA 的 S2MM 通道把视频流写入 DDR，软件配置帧地址、行跨度与图像尺寸。每个 RGB888 全帧为 $1280 \times 720 \times 3$，即 2,764,800 字节。摄像采样帧与包含文字、框线的显示帧分开管理，使显示叠加不会直接污染网络输入。识别前由 PS 安全取得完整帧，并复制到 PREP 专用输入缓冲。

摄像寄存器配置经 PS GPIO 的 EMIO SCCB 链路完成，用于设置传感器模式；像素数据沿并行视频接口传输。camera_bridge_ctrl 提供控制和状态寄存器，bridge_control_0 在配置和时钟稳定后依次释放视频桥复位并放行流，降低启动时半帧与不稳定同步对采集的影响。

2.2.3.3 图像预处理模块

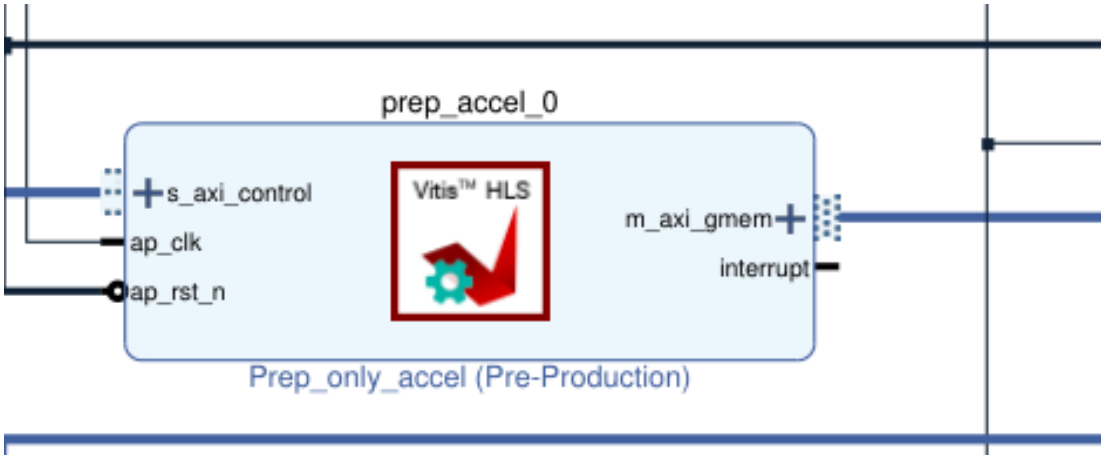


图 9 PREP 硬件预处理局部

prep_accel_0 是以 HLS 实现的 PREP 专用 IP，当前顶层为 prep_only_accel。PS 通过 s_axi_control 设置输入输出缓冲地址、步长、模式和启动位；IP 通过 m_axi_gmem 主动读写 DDR，经 prep_regslice 和 prep_mem_intercon 接到 HP1。

该模块完成检测图生成、灰度处理、水平背景计算和笔迹图计算，将规则的逐像素运算放在 PL 中执行。PS 中的 C 程序继续完成有效笔迹定位、无效输入拒绝、归一化、保持宽高比的缩放及居中，最终形成黑底亮笔迹的 64 × 64 UINT8 输入。预处理输出属于识别数据，屏幕上的绿色识别区和黄色引导线属于界面提示。

模式	当前功能
0	由原帧生成检测图并完成后续硬件预处理，正式应用使用
1	从已有检测图开始，执行后续灰度 背景与笔迹处理
4	身份查询，返回 0x50525001，不访问 DDR

模式	当前功能
其他模式	拒绝请求；当前 IP 不承担旧版 CNN 尾部计算

表 4 PREP 工作模式

PREP 已与 CNN 尾部计算分离，因此不能用旧版 mode 3 的卷积或全连接功能解释该实例。图中保留 interrupt 端口，但当前顶层没有接入 PS 中断入口，正式驱动采用完成状态轮询。硬件预处理与 PS 后处理之间需要保持数据布局和缓存一致，才能让同一帧得到可重复的网络输入。

2.2.3.4 统一 RTL 摄像识别网络

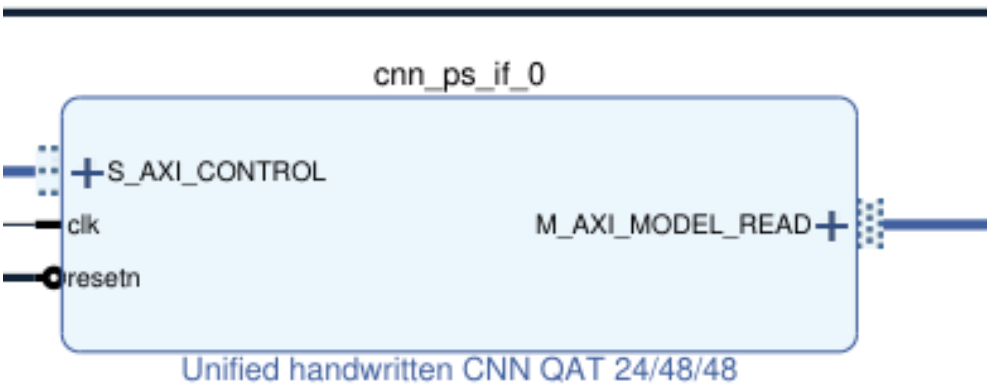


图 10 统一 RTL CNN 接口局部

cnn_ps_if_0 沿用历史实例名，当前内部为 cnn_rtl_unified 1.0，已经包含摄像识别的完整网络。输入为 1×64×64，模型通道配置为 24 48 48，输出 22 个有符号 32 位分类分值。一次 START 即可依次执行全部计算，中间特征保存在片内 RAM，不需 PS 在各层之间反复接力。

计算阶段	主要操作	输出张量或规模
Conv1 与 Pool1	3 × 3 卷积 1 → 24，量化 ReLU 饱和，2 × 2 池化	24 × 32 × 32
Conv2 与 Pool2	3 × 3 卷积 24 → 48，量化 ReLU 饱和，2 × 2 池化	48 × 16 × 16
Conv3 与 Pool3	3 × 3 卷积 48 → 48，量化 ReLU 饱和，2 × 2 池化	48 × 8 × 8
FC1	3072 维展开向量 → 48，量化 ReLU 饱和	48

计算阶段	主要操作	输出张量或规模
FC2	48 → 22，保留有符号 INT32 分值	22 个 logits

表 5 统一 RTL CNN 计算阶段

三层卷积均采用步长 1、边缘补零 1；池化为 2×2、步长 2。QAT 量化模型将浮点计算关系转换为整数乘加、定点缩放和饱和约束。FC2 不再做 ReLU，保留类别间的有符号分值供 PS 解释。

S_AXI_CONTROL 同时承载寄存器、4096 字节输入窗口和结果窗口；M_AXI_MODEL_READ 经 HP1 读取 DDR 中尾部权重与偏置。Conv1 与 Conv2 参数位于前端 ROM，尾部参数按需加载并缓存，尾部阶段共享量化资源。DONE 在计算与相关读事务安全结束后提交，PS 读取结果、确认当前请求仍有效，再执行 ACK。

2.2.3.5 显示与 HDMI 输出

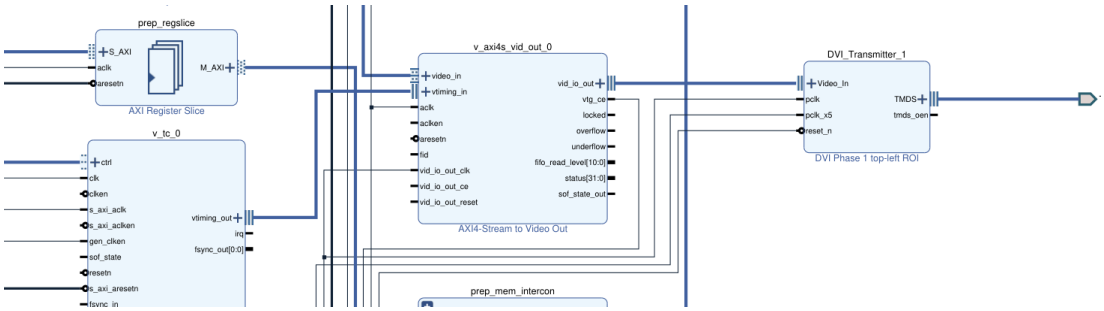


图 11 视频输出 时序与 TMDS 局部

系统的分子三维模型、文字说明与预览界面由 PS 软件渲染到 DDR 显示缓冲。axi_vdma_0 的 MM2S 通道经 HP0 读取显示帧，形成 AXI4 Stream 视频流；v_axi4s_vid_out_0 配合 v_tc_0 的时序，将其还原为原生视频；DVI_Transmitter_1 完成 TMDS 编码和串行输出，经 TMDS_0 连接显示器。

v_tc_0 是视频时序控制器，负责行场同步、消隐和有效显示区等时序，不生成界面内容。v_axi4s_vid_out_0 内部 FIFO 用于衔接 AXI 与像素时钟，locked、underflow 和 overflow 等信号可辅助定位供数或时序问题。DVI_Transmitter_1 负责传输编码，不能仅因历史描述中含 ROI 就将整个界面绘制归入该 IP。

当前输出分辨率为 1280×720，像素时钟为 74.25 MHz，TMDS 使用 371.25 MHz 的五倍像素时钟。clk_wiz_0 生成这两路时钟。371.25 MHz 是串行化相关时钟，不是图像刷新率，也不是 CNN 的工作频率。

摄像模式显示采集预览及识别区域；语音模式显示采音和识别状态，不保留摄像识别的大绿框。识别成功后，右侧统一显示该物质的名称、分子式、性质及球棍模型。三维交互由网页操作和 PS 状态控制，PL 显示链路持续输出软件生成的画面。

2.2.3.6 音频采集与语音接口

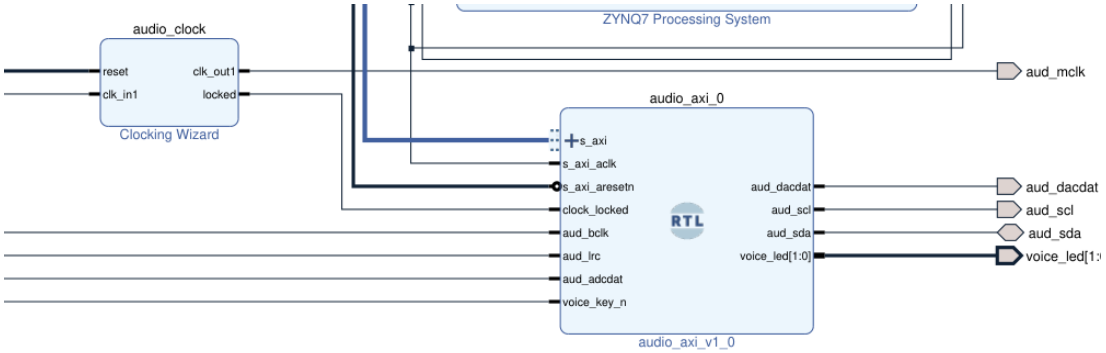


图 12 音频采集与音频时钟局部

audio_axi_0 对应自有 audio_axi RTL，负责连接板载 WM8960、接收 I2S 音频、进行滤波抽取并提供 AXI 可读 FIFO。audio_clock 由系统时钟生成 12 MHz 音频主时钟，其 locked 信号参与采集启动判断。

接口或数据	作用
aud_scl 与 aud_sda	通过 I²C 配置 WM8960 的音频通道和工作模式
aud_bclk aud_lrc aud_adcdat	分别为 I2S 位时钟 声道时钟和 ADC 串行数据
aud_mclk	向音频编解码器提供主时钟
PCM FIFO	48 kHz 采样经 FIR 三倍抽取，形成 16 kHz 单声道 PCM16；深度为 4096 样本
AXI 控制接口	PS 配置采集 读取状态与样本，并观察溢出和丢样

表 6 音频接口与数据

4096 个样本在 16 kHz 下约对应 256 ms 音频，因此软件应持续排空 FIFO，避免样本积压。之后的语音活动检测、特征提取和神经网络推理由 PS 中的 C 程序完成，audio_axi_0 不执行摄像 CNN，也不直接输出化学名称。

原图仍保留 voice_key_n 和 voice_led 端口，说明硬件具有按键与状态接口；本项目的主要操作入口为电脑网页。网页触发与底层按键引脚属于不同层次，不能仅凭 BD 中存在该端口，判断应用一定启用了按键触发。当前采集链路将 aud_dacdat 保持为静音输出，语音输入使用开发板麦克风。

2.2.3.7 时钟 复位与启动控制

时钟域	频率或来源	服务的主要模块
FCLK0	设计 100 MHz，运行策略 100 MHz	GP0 CNN PREP HP1 摄像写入侧 音频 AXI 监视器与 ILA
FCLK1	实现约束 150 MHz，应用策略 100 MHz	HP0 出口 VDMA 显示读取侧及 AXI 视频输出
视频像素时钟	74.25 MHz	VTC 原生视频输出 TMDS 像素输入
视频五倍时钟	371.25 MHz	TMDS 串行化
音频主时钟	12 MHz	WM8960 MCLK
摄像像素时钟	由 OV5640 输入	摄像采集与视频桥原生输入域

表 7 时钟域与模块分工

rst_ps7_0_100M 与 rst_ps7_0_150M 分别对相应时钟域提供同步复位。复位释放需结合时钟稳定状态，且应区分低有效 aresetn 与高有效 reset。摄像桥另有局部复位门控，音频模块也有自己的启动条件。实例名称中的 150M 保留自实现配置，应用运行时钟应以软件设置及硬件读回为准。

2.2.3.8 摄像监视器与在线调试

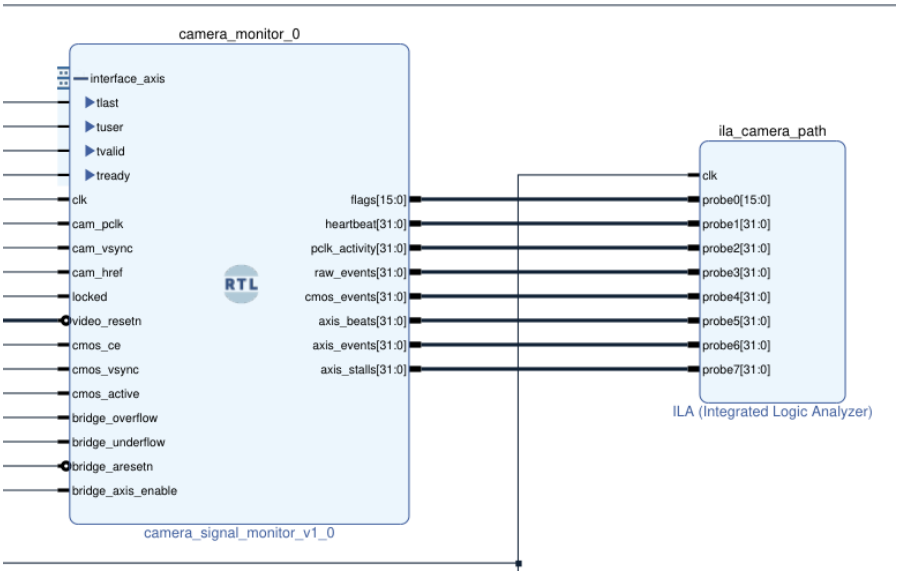


图 13 摄像监视器与 ILA 局部

camera_monitor_0 旁路观察摄像原始同步、整理后视频、AXI 握手和桥状态，输出心跳、像素时钟活动、帧行事件、成功数据拍和阻塞计数。ila_camera_path 采样这些探针，帮助区分传感器未启动、桥未放行、同步异常与下游背压。监视器和 ILA 不修改识别输入，也不参与卷积或显示绘制。

2.3 软件系统介绍

2.3.1 软件整体介绍

系统软件由开发端模型工具、板端应用程序和 PC 端网页组成。开发端采用 Python 完成自制数据集处理、摄像识别网络训练、量化感知训练及整数参数导出，为硬件部署提供模型参数、类别映射和参考输出。软件总体架构与数据流如图 14 所示。

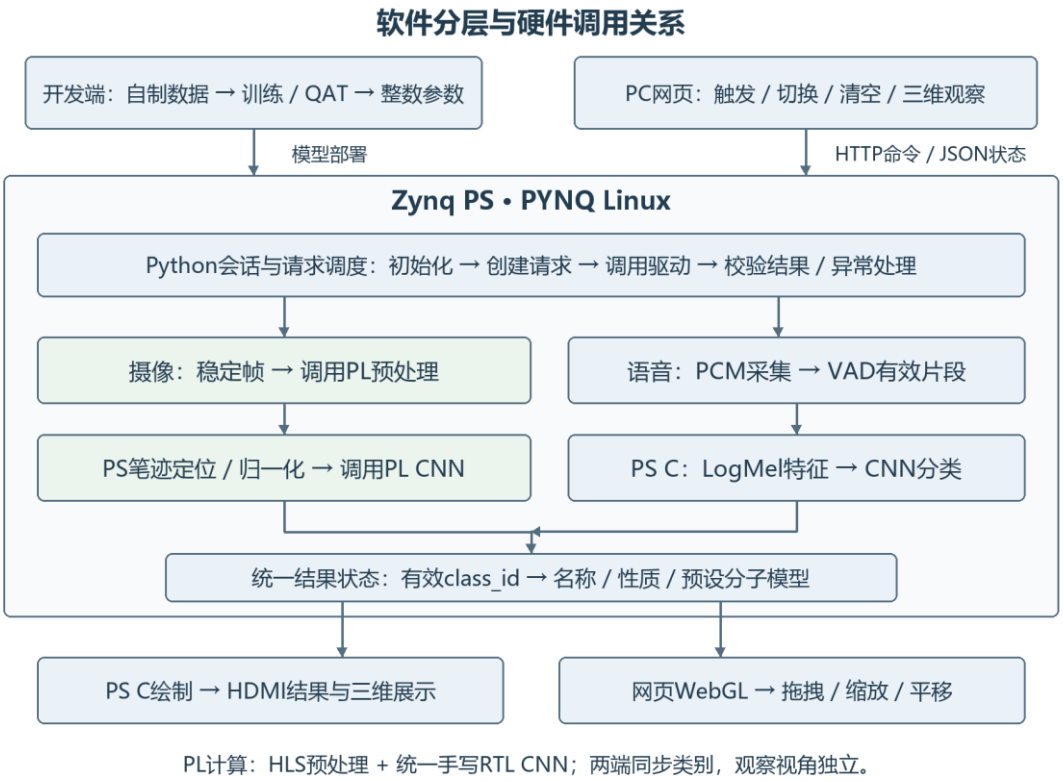


图 14 系统软件总体架构与数据流

板端软件运行于 Zynq PS 侧的 PYNQ Linux 环境，采用 Python 与 C 协同实现。Python 负责初始化、硬件调用、请求调度、状态管理和网页服务；C 负责笔迹后处理、语音特征提取与推理、分子三维渲染及界面绘制。摄像识别通过软件依次调用 PL 预处理与统一 RTL CNN，中间由 PS 完成笔迹定位和尺寸适配；语音识别由 PS 端 C 程序完成。

摄像、语音和手动选择共用类别映射，依据 22 类预设有机物的类别编号查询名称、分子式、性质及分子模型。HDMI 端显示摄像预览、结果信息和旋转球棍模型；PC 端通过局域网访问开发板网页，完成识别触发、模式切换、清空及状态查询，并支持分子模型拖拽旋转、缩放、平移和视角复位。两端同步分子类别，网页观察视角可独立调整，界面如图 15 所示。识别推理在开发板本地完成。



（a）摄像识别与分子三维交互



（b）语音识别结果状态

图 15 网页交互与分子三维观察界面

注：截图用于功能演示，单次计时不作为性能统计；性能指标见表 1。

2.3.2 软件各模块介绍

软件按会话管理、业务调度、算法处理、硬件访问和交互展示划分模块。上层通过统一请求状态组织摄像与语音业务，底层驱动负责寄存器交互、缓冲区和错误处理，各模块关键函数及输入输出见表 8。

软件模块	关键调用	关键输入	关键输出
初始化与会话	session.check() session.start()	工程配置、模型与硬件文件、启动方式	app 对象、初始化及资源状态
采集与预览	capture.acquire() app._step()	帧缓冲、交接策略	稳定图像帧、显示更新
图像预处理	prep.run() page.postprocess()	1280×720 BGR 图像	64×64 UINT8 图像、预处理状态
RTL CNN 调用	unified.infer()	image、transfer 及已加载模型	22 个 INT32 logits、predicted、硬件状态
语音交互	voice_once() _voice_run()	音频输入配置、PCM 数据	类别、置信信息或拒识原因
请求与结果	requests.begin() publish()/reject()	请求标识、模式版本、识别结果	有效类别及请求状态
显示与网页	_render() snapshot()/command()	类别编号、应用状态、网页命令	HDMI 画面、JSON 状态与网页展示
正常停止	session.stop()	当前会话及资源所有权	停止报告、资源释放结果

表 8 软件模块与关键输入输出

2.3.2.1 初始化与请求调度

PynqSession.check()检查工程文件、模型及 bit/HWH 配对关系；start()根据 mode 与 clean_boot 建立唯一硬件会话，初始化摄像、显示、预处理、CNN 和音频，返回应用对象 app。应用通过 requests.begin()创建请求，再依据当前模式调用 recognize()或 voice_once()。主要调用顺序为“工程校验→取得会话所有权→检查或加载配套硬件→初始化驱动和模型→启动预览与网页服务→接收业务请求”。

2.3.2.2 摄像采集与预览

ParkedCapture.acquire()在检查新帧事件、写者切换和 DMA 状态后返回稳定图像帧。预览_step()依次取帧、取得后台显示缓冲、渲染并提交显示；正式识别按需触发。针对预览等待较长的问题，将第二次新帧事件等待与 park 切换确认重叠，保留原有帧交接和所有权条件。该调整只涉及 PS 预览调度，不改变识别计算链或 Block Design。

2.3.2.3 图像预处理调度

recognize()首先建立摄像请求并取得 1280×720 BGR 图像帧，调用 prep.run()由 PL 完成 ROI 缩小、灰度转换、滤波及背景校正，再由 page.postprocess()调用 PS 侧 C 程序完成连通区域筛选、笔迹定位、灰度归一化、等比例缩放和居中。输出 image 为 64×64 UINT8 灰度图，共 4096 字节；预处理报告中的 code 为 0 时输入有效，空白、触边或信号不足则返回相应拒识原因。

2.3.2.4 统一 RTL CNN 驱动

UnifiedCnn.infer(image, transfer)检查硬件身份、模型就绪及事务空闲状态，写入并核对 4096 字节输入，通过一次 START 启动整网。完成后核对事务序号、完成标志、数据计数和错误状态，读取 22 个有符号 INT32 分值 logits，并由 argmax 得到 predicted。结果发布前再次检查请求有效性，防止清空或切换模式后的旧结果重新显示；超时或硬件异常进入恢复流程。完整摄像调用关系如图 16 所示。

2.3.2.5 语音识别

voice_once()建立单次语音请求并启动后台任务，_voice_run()组织音频采集、语音活动检测、特征提取和模型推理。采集的 16 kHz、16 位 PCM 数据经环境噪声估计和有效片段截取后，生成对数 Mel 特征，由 PS 端 C 模型分类，再依据置信度 confidence 及类别分差 margin 判断是否接受。有效输出包含 class_id；静音、低分或无效输入返回拒识原因。结果经当前请求校验后汇入统一显示状态，流程如图 16 所示。

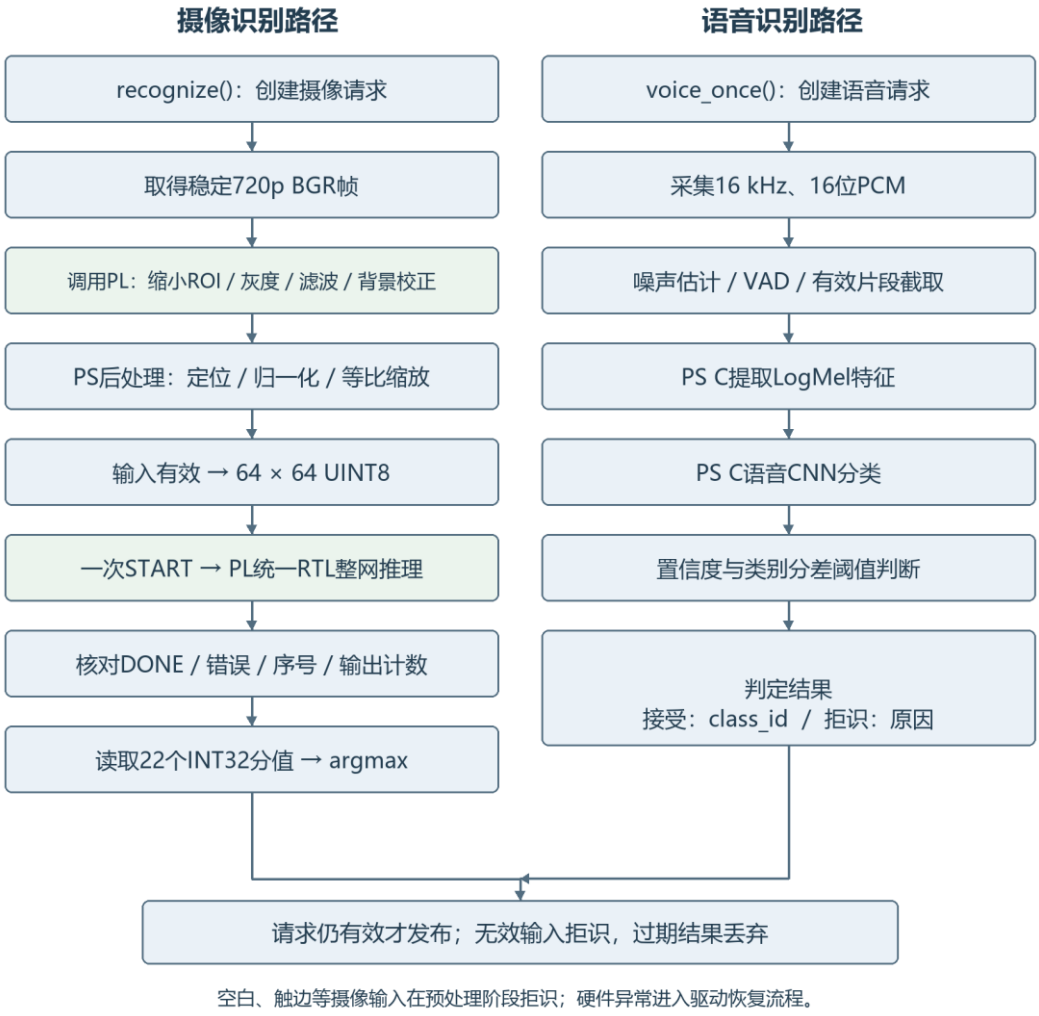


图 16 摄像与语音识别的软件调用流程

2.3.2.6 结果管理与双端三维展示

系统依据有效 class_id 调用预设分子模型，并查询名称、分子式及性质信息。HDMI 端由 PS 侧 C 程序完成球棍模型旋转、投影和界面合成，再通过显示缓冲提交画面；网页端在浏览器中完成三维渲染及鼠标交互。分子类别同步后，网页观察视角独立变化，不回传为识别结果。

网页通过 HTTP 命令控制开发板，并轮询获取 JSON 状态。服务端的 command() 检查命令及状态版本，snapshot()提供当前结果快照；server_id、mode_epoch 和 request_id 分别标识服务实例、模式版本及请求编号，用于拒绝过期操作。涉及硬件的任务在统一应用锁和资源所有权约束下执行，状态查询与耗时任务分离，减少录音或推理期间网页失去响应的情况。

2.3.2.7 异常处理与正常停止

正常退出由 session.stop()统一组织：先停止接收网页命令并等待相关任务退出，再取消请求、停止音频及预览、关闭硬件访问，确认安全后释放缓冲区和会话资源。停止报告记录 errors、retained_buffers、dma_owned、线程状态以及 active_bos 等信息；若无法确认硬件停止，则保留资源及错误信息，避免提前释放仍被访问的内存。结果发布与退出控制如图 17 所示。

实板停止报告显示 errors 为空，13 个缓冲区全部释放，active_bos 与 active_bytes 均为 0，DMA、预览、语音及网页线程正常退出，三维模块和请求控制器关闭，验证了本轮运行结束后的资源释放。



图 17 结果发布与正常停止流程

2.4 软硬件划分依据与接口设计

任务划分同时考虑运算规则性、控制复杂度、访存方式及器件资源。PL 承担固定节拍的数据接收、逐像素处理和整数乘加；PS 承担数据相关分支、请求调度及显示内容生成。表 9 说明各项选择，具体数据通路仍以图 4、图 14 及图 16 为准。

任务	部署位置	划分依据与主要接口
视频与音频接收	PL	摄像像素、I2S 及视频时序要求连续处理；通过 AXI 视频流、VDMA 和音频 FIFO 向 PS 提供数据
图像预处理	PL + PS	PL 处理规则的 ROI 缩小、灰度、滤波与背景；PS 处理连通区域、拒识、等比缩放和居中，输出 64×64 UINT8
摄像 CNN	PL 手写 RTL	固定网络便于定点化与输出通道并行；一次 START 完成整网，输出 22 维 INT32 分值
语音与业务控制	PS	复用已验证的 C 前端与语音模型；处理 VAD、阈值、取消和请求身份，避免与摄像 CNN 混用参数和张量
显示与三维交互	PS / PC + PL 输出	PS 绘制 HDMI 界面与球棍模型，PC 浏览器渲染交互模型；PL 负责显示帧传输和视频输出时序

表 9 软硬件划分依据与主要接口

摄像 CNN 输入为按行连续的 4096 字节 UINT8 图像，方向、灰度规则和类别顺序均随模型冻结。PS 通过 AXI-Lite 输入窗口写入数据，启动前检查硬件身份及空闲状态；完成后检查 DONE、错误、序号和输出计数，读取 22 个有符号 INT32 分值，再确认请求仍有效并执行 ACK。尾部参数由只读 AXI 主接口经 HP1 从 DDR 加载，层间特征保留片内，不经 PS 逐层搬运。

摄像源帧与绘制后的显示帧分开管理，缓冲交接以完整帧、DMA 状态和所有权检查为依据；CPU 与 PL 共享的 DDR 缓冲按读写方向执行对应缓存同步。网页采用 HTTP 命令与 JSON 状态轮询，服务实例、模式版本和请求编号用于拒绝过期操作；两端同步分子类别，浏览器视角独立，不回写为识别结果。

2.5 模型训练与定点部署原理

摄像训练与开发验证数据均由团队采集制作，保存处理后数据、类别映射、训练检查点及导出清单。训练采用浮点模型与量化感知微调；QAT 前向模拟部署时的整数权重、Q30 缩放、舍入与饱和，反向使用连续近似梯度，使模型适应固定运算规则。量化训练的基本思路可参考整数推理相关研究，本项目的具体数值规则以冻结源码和参考输出为准。

三层卷积对输入与 3×3 权重逐项乘加并加入偏置，量化后执行 ReLU、0 至 127 饱和及 2×2 最大池化；FC1 采用同类激活量化，FC2 保留 22 维有符号 INT32 分值。设累加结果为 a、非负 Q30 系数为 M，当前整数参考采用下列规则，半值向远离零的方向舍入：

$$p = a \times M;\quad q = \mathrm{sign}(p) \times \mathrm{floor}((|p| + 2^{29}) / 2^{30})$$

$$y = \min(127, \max(0, q))$$

导出过程固定权重、偏置、缩放系数及类别表，卷积和 FC 的 INT32 累加范围由参考程序检查。验证依次核对网络输入、各层累加、激活、池化及最终分值，优先定位第一处差异。910 张开发验证图像用于模型能力评估；固定输入仿真与 132 组实板记录用于部署一致性和性能检查，二者承担不同任务。

第三部分　完成情况及性能参数

3.1 整体介绍

本系统以 Zynq-7020 开发板为核心，集成摄像头、音频采集模块、HDMI 显示器与浏览器上位机，支持手写有机物结构简式识别和化学名称语音识别。系统根据识别结果展示分子名称、分子式、相关性质及三维分子结构，并实现板端显示与网页端的联动，为化学知识学习和分子结构观察提供交互式展示。系统整体实物如图 18 所示。



(a) 系统正面实物图　　(b) 系统斜侧面实物图

图 18 系统实物

3.2 特性成果

3.2.1 系统功能完成情况

系统已完成摄像、音频采集、识别计算、分子三维展示及网页交互的整机集成。两种识别方式共用类别映射，将有效结果关联到名称、分子式、性质及预设分子模型。典型界面见图 1 和图 15，整机连接见图 18，各项功能的板级联调结果如表 10 所示。

功能项目	实现内容	验证结果
摄像识别	实时预览、笔迹预处理及 22 类预设有机物分类，显示名称、分子式与性质	通过
语音交互	单次触发化学名称识别，处理静音、未知词及低置信度输入	通过
三维展示	HDMI 显示旋转球棍模型；网页支持拖拽旋转、缩放、平移与视角复位	通过
双端交互	支持模式切换、识别触发、类别选择、清空及状态同步	通过
输入与请求管理	无效图像拒识、取消请求，阻止清空或切换模式后的旧结果重新显示	通过
正常停止	结束采集、识别和显示任务，关闭网页服务并释放硬件缓冲区	通过

表 10 系统功能完成情况

系统当前支持 22 类预设有机物，三维结构由有效类别编号调用对应模型，网页观察视角独立调整。

3.2.2 自制数据与部署数值验证

摄像识别网络的训练与验证数据集均由团队自主采集制作，形成从原始数据、训练与量化到参数导出的完整工程链路。量化模型在 910 张自制开发验证图像中正确识别 826 张，准确率为 90.77%。该结果表征当前模型在开发验证集上的分类能力。

部署验证采用固定定点规则和逐层整数参考对拍。第三层四路并行优化保留各输出通道的累加顺序，完成 66 组第三层专项 RTL 回归及 4 次完整网络逐层对拍，输出均与整数参考一致。实板 132 组配对测试的 22 维整数输出也全部匹配冻结参考，验证了所测输入上的数值一致性。

注：132 组记录来自 44 张固定输入重复运行 3 轮，用于性能及一致性验证，不计为 132 张独立图片，也不用于更新上述准确率。

3.2.3 同板 PS 软件与 PL RTL 性能对比

为评价手写 RTL 加速器的实际收益，在同一块 Zynq-7020 开发板上，对相同定点 CNN 分别采用 PS 单线程优化 C 和 PL RTL 实现。PS 程序采用 GCC 9.3.0、-O3 及 Cortex-A9/NEON 向量化编译选项，PS 实际时钟约 766.67 MHz，PL CNN 运行时钟为 100 MHz。

选取 22 类各 2 张固定输入，预热 5 组后交替安排两种实现的执行顺序，运行 3 轮，共获得 132 组配对记录。表 11 给出实测结果。

性能指标	PS 单线程优化 C	PL RTL 完整调用
平均推理耗时	48.933 ms	**26.680 ms**
中位推理耗时	48.816 ms	26.621 ms
P95 推理耗时	49.241 ms	26.819 ms
相对 PS 软件加速比	1.00 倍	**1.83 倍**
相对 PS 软件耗时降低	基线	**45.48%**

表 11 同板同定点模型的 PS 与 PL 推理性能对比

注：P95 表示 95%的测量记录不超过该耗时。PL 计时包含输入传输与校验、启动、等待、结果读取及 ACK；两种实现均不计入摄像采集、图像预处理、显示和初始化。

在计入驱动与数据交互开销后，PL 完整调用平均耗时仍由 PS 软件的 48.933 ms 降至 26.680 ms，实现 1.83 倍加速、耗时降低 45.48%。该结果体现摄像 CNN 推理环节的收益，不等同于整机交互加速比或摄像预览帧率。

3.2.4 并行优化与整机资源代价

优化首先解决功能与资源收敛：保留已验证前端，逐步将第三层及全连接统一到手写 RTL，通过尾部参数 DDR 按需加载、特征片内保存和公共加载 / 量化资源共享，减少重复计算模块及 PS 层间交接。前期整机由混合实现的 131 个 BRAM、133 个 DSP，收敛为单路整网 RTL 的 128 个 BRAM、123 个 DSP；这是资源重构阶段的结果，最终四路版本 BRAM 为 128.5。

初版第三层卷积采用单路乘加复用，是整网主要耗时来源。优化后采用四输出通道并行，广播输入特征并更新四路累加结果；通过一块 RAMB18 缓存通道组权重，复用原有特征缓存与量化器，避免直接复制整套计算和存储模块。表 12 比较第三层单路与四路版本的完整整机资源。

指标	优化前单路 RTL	优化后四路 RTL	变化
完整调用均值*	66.558 ms	26.680 ms	耗时降 59.91%
LUT	27,563	27,921	+358
FF	36,329	36,726	+397
BRAM tile	128.0	128.5	+0.5
DSP	123	123	0

表 12 第三层并行优化前后的性能与整机资源对比

注：资源为同器件完整布局布线结果，包含摄像、显示、预处理、CNN、音频及调试模块。BRAM 以 36 Kb tile 为单位。*单路与四路耗时来自两次历史会话，用于观察优化趋势，约为 2.49 倍；不替代表 11 同会话 132 组 PS/PL 配对所得的 1.83 倍加速。

单路整网 RTL 虽已完成数值对拍，首次同板配对中 PS 优化 C 均值为 49.394 ms、单路 RTL 为 66.558 ms，尚未达到加速目标。因而后续不以“使用了 FPGA”代替性能结论，而是针对主要耗时层调整并行度和供数方式，再以表 11 所列条件重新评价。四路优化保留原累加顺序及公共量化器，用较小整机资源增量取得正向加速。

四路并行仅增加 358 个 LUT、397 个 FF 和 0.5 个 BRAM tile，DSP 数量保持不变。完整整机资源利用率见图 19，其中 BRAM 利用率为 91.79%。

Resource	Utilization	Available	Utilization %
LUT	27921	53200	52.48
LUTRAM	927	17400	5.33
FF	36726	106400	34.52
BRAM	128.50	140	91.79
DSP	123	220	55.91
IO	33	125	26.40
MMCM	2	4	50.00

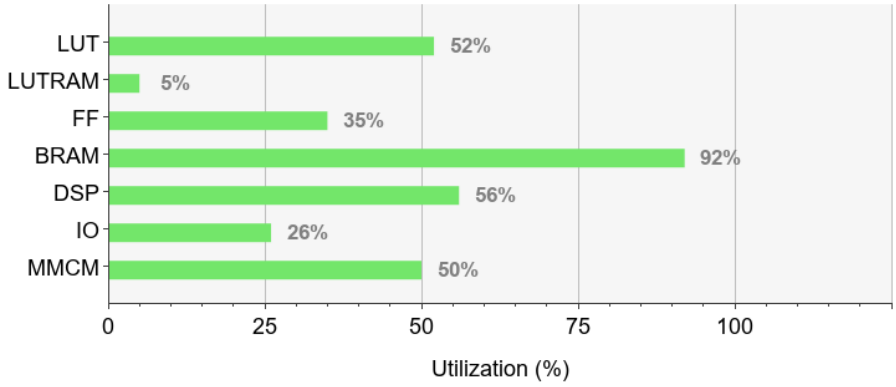


图 19 优化后系统布局布线资源利用率

当前布局布线报告中已约束路径的最差建立时间裕量 WNS 为＋0.119 ns，最差保持时间裕量 WHS 为＋0.011 ns，满足所设时序约束。片上存储是当前主要资源约束，系统通过尾部参数 DDR 按需加载、片上特征缓存和计算资源共享，在器件容量内完成整网 CNN 与视听交互模块的集成。

3.2.5 预览调度与安全退出

针对摄像预览等待较长的问题，将新帧事件等待与 park 切换确认重叠，在保留稳定帧交接及资源所有权检查的条件下减少重复等待。该优化仅调整 PS 预览调度，Block Design、模型和 CNN 计算链保持一致，实板使用反馈画面流畅性得到改善。

预览调度与 CNN 计算分别优化：前者减少帧交接的串行等待，后者减少定点网络计算耗时。帧率对照报告因 NumPy 对象 JSON 序列化错误及随后断电未完整保存，最终仅记录用户观察到的流畅性改善和正常停止证据，不据旧截图或主观感受填写优化后 FPS。修复报告保存与停止入口后，日常使用不再依赖自动对照结果。

系统已完成两项回放及摄像、语音、三维展示、网页交互和正常停止验证。一次实板停止报告显示 errors 为空，13 个缓冲区全部释放，active_bos 与 active_bytes 均为 0，DMA 所有权已释放，相关工作线程退出。退出时先停止硬件访问、确认安全后释放资源；无法确认停止时保留资源和错误记录，避免提前释放仍被访问的内存。

交付工程同步保留硬件与软件源码、配套 bit/HWH、模型参数及性能原始记录，提供文件校验和启动说明，便于按相同版本复现上述结果。

3.3 创新设计与工程贡献详解

本作品面向 Zynq-7020 的计算、存储与交互约束，将自制数据、定点模型、手写 RTL 及分子可视化整合，技术贡献体现在以下六个方面。

3.3.1 整网调度与层间特征驻留

摄像 CNN 从三层卷积至两层全连接由统一 RTL 控制，一次 START 完成整网计算并发布 22 维分值。中间特征按约定布局驻留片内，PS 负责输入、启动和结果处理，减少逐层调用及层间张量交接。

尾部模型参数经只读 AXI 从 DDR 按需装入，计算阶段复用局部缓存与公共量化资源。将参数供给、特征生命周期和整网调度协同设计，使完整 CNN 能够与采集、显示及音频模块在同一器件内集成。

3.3.2 输入广播与四通道并行供数

Conv3 将同一输入特征广播到四个输出通道，各路独立读取权重并累加；Pool2 特征保留一份，通道组权重进入局部缓存，计算后共享量化器。该结构围绕瓶颈扩展计算和供数，控制特征存储的复制开销。

相较单路整网版本，完整实现增加 358 个 LUT、397 个 FF 及 0.5 个 BRAM tile，DSP 保持 123。四路完整调用包含传输与控制开销，同板测试较 PS 优化 C 加速 1.83 倍。表 11、表 12 分别给出性能与资源依据。

3.3.3 定点规则冻结与分层验证

QAT、导出、整数参考和 RTL 统一输入布局、缩放、舍入及饱和规则。并行化保留各通道累加顺序，FC2 保留 INT32 分值；逐层张量和 22 维输出共同作为验收对象，使调度或并行改动引入的数值差异可以定位。

开发验证集评价模型分类能力，固定输入对拍检查部署是否符合冻结参考。第三层专项、整网仿真与板端配对形成分层证据，支持在优化硬件结构时核对所测输入的整数计算结果。

3.3.4 自制数据到部署参数的完整链路

摄像模型的训练与开发验证数据均由团队采集制作，并保留类别映射、预处理规范、训练检查点、QAT 及导出源码。数据构建与硬件输入共同围绕纸面结构简式展开，最终以 64×64 黑底亮笔迹图像作为接口，使样本、模型和部署格式可以相互追溯。

自主数据的工程价值在于便于按具体误识别样本检查光照、书写、定位或模型环节，并定向补充数据。当前 90.77%对应 910 张开发验证图像，适用范围仍为 22 类预设有机物；后续增类或评价新的书写场景时，可以沿同一流程重新训练、导出和核验。

3.3.5 视听输入与双端三维交互

摄像识别结构简式与语音识别化学名称两条通路，共用有效类别及物质信息映射，将纸面表达、名称和空间形态连接起来。PL 承担规则图像计算和音频前端，PS 分别完成图像后处理、语音推理及业务控制，使两种输入能够接入统一的交互流程。

HDMI 端由 PS 绘制球棍模型并输出旋转画面，网页端采用 3Dmol.js/WebGL 提供拖拽旋转、缩放、平移及视角复位。两端同步分子类别，浏览器观察视角独立调整。识别结果因此可以直接进入空间观察，适合课堂展示和分子结构讲解；三维模型由预设类别调用。

3.3.6 将请求有效性纳入系统可靠性设计

系统为服务实例、模式版本和请求分配身份，结果发布前再次核对；切换模式、取消或清空后，迟到的识别结果不会重新覆盖当前界面。正常停止依次结束命令、任务和硬件访问，确认 DMA 与缓冲区使用结束后再释放资源，将交互状态与硬件生命周期联系起来。

配套交付保留同版本源码、bit/HWH、模型、原始结果及校验配置，并提供从解压到安全停止的操作步骤。其工程特色是把识别效果、计算性能、状态一致性和可复现性作为整体交付目标；既有功能验收、停止报告及文件校验分别提供相应依据。

第四部分　总结

4.1　可扩展之处

本系统可从识别能力、硬件加速和应用交互三个方面继续扩展。在识别能力上，可增加化学物质类别，补充不同书写风格、光照条件和语音环境的数据，增强复杂场景下的识别与拒识能力。在硬件加速上，可进一步分析语音特征提取、语音网络和图像预处理的耗时，将适合并行处理的部分迁移至 PL 端，并结合定点量化、存储复用和流水线设计控制资源开销。在应用交互上，可依托 PYNQ 与网页接口增加识别记录、分子结构对比和教学演示功能。后续扩展应同步核对模型、驱动与硬件接口，并通过同条件测试评价性能和资源变化。

当前范围与后续验证应分开说明：22 类固定对象的分类及预设三维模型适用于本报告的教学演示任务；增加未知类别、任意结构重建或验证独立书写者泛化能力，需要新的数据与评估。后续扩展同时考虑 BRAM 余量、端到端交互时延及外设接口约束，不能仅依据网络推理速度推断整机体验。

4.2　心得体会

通过本项目的设计与制作，我们认识到，嵌入式智能系统的实现不仅取决于模型本身，还依赖采集、存储、计算、控制和显示等环节的协同。项目将手写化学式识别、化学名称语音识别与分子三维展示连接起来，使我们对从输入数据到应用结果的完整处理过程有了更具体的理解。

在软硬件划分方面，我们逐步认识到，并非所有功能都适合直接改写为 Verilog。摄像数据接收、视频传输和卷积乘加具有较强的规律性，适合交给 PL 处理；任务调度、语音识别、网页交互和三维渲染则保留在 PS 端，以兼顾开发效率与资源限制。统一 RTL 摄像 CNN 通过一次启动完成网络计算，减少了 PS 逐层调度和中间特征交接，但完整识别过程仍包含采集、预处理和结果发布，因此评价加速效果时必须明确测量范围。

在硬件联调方面，我们体会到，接口名称正确并不意味着数据一定能够正确传输。摄像图像需要经过像素拼接、视频协议转换和 VDMA 写入，任何一处时钟、复位、帧边界或握手异常，都可能影响后续处理。AXI 传输必须满足 VALID 与 READY 握手条件，遇到背压时还需保持数据稳定。理解这些时序关系，比仅观察屏幕是否出现画面更有助于定位问题。

在模型部署方面，训练得到的模型不能直接等同于可用的板端实现。输入尺寸、数据布局、量化缩放、饱和处理及类别顺序都需要保持一致。固定输入回放和软件参考结果为核对提供了依据，也提醒我们将离线验证、实板正确性和真实场景表现分别评价，不能用单次识别成功代替完整测试。

应用完善同样需要关注细节。例如，摄像模式需要识别区域提示，而语音模式应隐藏摄像框线；切换模式或取消任务后，应避免旧结果覆盖当前状态；退出时，还需先停止硬件访问，再释放缓冲区。这些细节直接影响系统的稳定性与使用体验。

大模型在本项目中用于方案讨论、代码与报告辅助；设计人员保留目标设定、方案审核、实板操作和验收责任。关键协作记录及依据实测修正建议的过程放在报告最后，说明哪些建议被采用、哪些判断被纠正，以及结果能够支持的结论范围。

第五部分　大模型协作记录

5.1 协作方式与记录说明

团队确定模型、接口和资源约束，向大模型提供代码及实测结果，讨论候选方案；再通过整数参考、RTL 仿真、综合实现和人员实板操作核验。每次迭代先明确需要保持的数值规则和计时范围，再依据结果决定是否采用建议。早期两层模块、统一 RTL 基础组件与当前整机属于不同阶段，其仿真、综合和板测结果分别说明。大模型辅助分析与实现，最终设计取舍和验收由团队完成。

5.2 资源约束下的 PL 迁移范围与软硬件划分

团队提问：希望通过 Verilog 实现关键计算并部署到 PL，提高识别响应速度。应如何按计算密度、复用程度和控制复杂度确定迁移范围与优化优先级？在 Zynq-7020 上，仅检查 Slice 占用是否足以判断方案可行？如何同时评估 BRAM 容量及端口、DDR 供数、DSP 映射和时序余量，并合理安排语音及双端三维交互？

模型建议摘要：将卷积乘加、规则像素处理和连续视频传输交给 PL，将动态分支、应用调度与交互逻辑保留在 PS。资源评估分别查看 LUT、FF、占用 Slice、BRAM 及 DSP，避免把 Slice 占用率等同于 LUT 利用率。并行度应与权重和特征的存储布局、访问端口及供数能力匹配，最终以布局布线后的资源和时序确认可行性。

模型建议摘要：图像预处理按步骤拆分：PL 执行 ROI 缩小、灰度、滤波与背景校正，PS 执行笔迹定位、拒识、归一化和尺寸适配。HDMI 画面由 PS 生成，PL 承担显示帧传输与视频时序；网页模型由浏览器渲染，两端同步有效类别，网页观察视角在本地调整。语音保留独立的 PS C 模型与输入格式，是否迁移到 PL 由后续瓶颈测量决定。

实施与方案取舍：工程经历 SPEED 阶段的图像链加速、统一 RTL 整合及 Conv3 并行优化，迁移范围逐步收敛到摄像识别的数据与计算通路。团队依据源码和 Block Design 逐项核对职责，将 PS 侧三维渲染、显示帧传输及 TMDS 输出分开说明；网页使用 3Dmol.js/WebGL 提供旋转、缩放与平移，HDMI 展示 PS 生成的球棍模型。

验证结果：当前整机布局布线占用 128.5/140 个 BRAM tile、11884/13300 个 Slice；Slice 与片内存储均需纳入预算，不能仅依据剩余 DSP 继续扩展计算通路。已有摄像、语音、三维和网页联调确认上述部署结构，优化目标据此同时考虑计算延迟、层间搬运、存储开销及实现风险。

5.3 Pool1 特征缓存复用与两层卷积前端优化

团队提问：早期两层 CNN 的调度未保存完整 Pool1 特征，第二层不同输出通道需要反复触发第一层计算。如何区分卷积核内乘法并行、输入通道并行和输出通道并行，定位重复计算的开销？在保持零填充、网络运算和最终输出顺序的前提下，能否以有限 BRAM 保存可复用特征，同时防止缓存覆盖和跨通道数据错位？

模型建议摘要：保存完整的 24×32×32、8 位 Pool1 特征，共 24KiB，使第一层每帧只执行一次。Conv2 以两路输出通道并行为初始配置，共享输入窗口，各自保留权重和部分和，不通过复制整份特征图获取带宽。并行池化结果先暂存，再按 CHW 即通道、行、列顺序输出。

模型建议摘要：按全局行号 g=IC×32+Y 将缓存分成三个 bank，bank=g%3、行内地址由 g/3 和 X 组成，使同一通道的相邻三行可同时读取。边界补零、bank 选择、坐标及 valid 随 RAM 读延迟同步。控制分为写入、写满锁定、读取和释放阶段，收齐 Pool1 后才启动 Conv2，消费结束前不允许覆盖。

实施与方案校正：团队将首层反复重算改为完整特征缓存复用，Conv2 默认采用两输出通道并行。资源审查同时纠正“乘法器实例数等于 DSP 数量”和“有效字节数等于实际 BRAM 需求”的估算：该阶段部分乘法器由 LUT 实现，浅行 RAM 还会产生容量取整与利用不足。实现重点因此放在存储组织、调度、有效标志与输出重排，并检查连续多帧和边界输入。

验证结果：早期独立交付模块在 Vivado 2024.1 原生 IP 重建工程中，完成原样本、全零及四边非零输入的连续三帧、六阶段逐值核对，帧间不复位。三帧周期分别为 759468、759550 和 759506，按 100MHz 折算约 7.595ms/帧；综合为 3931 LUT、2067 FF、47.5 个 BRAM36 等效块及 6 个 DSP，完整 Pool1 缓存占 6 个 BRAM36。

阶段边界：上述数据仅属于早期两层卷积前端，其参数版本与当前三层模型不同，不包含完整 CNN、PS、显示和音频，也未形成该模块的布局布线或板测结论。缓存复用方法被后续工程继承，早期仿真耗时不与当前整机 26.680ms 调用直接比较；未经同条件实测的旧调度估算不计算为加速比。

5.4 定点 CNN 迁移的数值一致性控制

团队提问：将已经冻结的摄像 CNN 迁移为统一手写 RTL 时，我们希望保留现有量化模型的分类行为，同时为后续并行优化建立可复用的验收条件。如何把输入布局、乘加位宽、Q30 缩放、负数半值舍入、ReLU、饱和与池化顺序定义为可检查的数值规则？仅比较最终类别可能掩盖哪些中间误差？怎样区分模型分类误差与硬件部署误差，并控制逐层验证的范围？

模型建议摘要：先冻结同一版本的预处理输出、权重、偏置、缩放系数和类别顺序，以整数参考作为硬件实现的判定依据。累加及缩放乘积应保留足够的中间位宽，明确舍入与截位位置；对于本项目的 Q30 规则，半值向远离零的方向舍入，再执行 ReLU、0 至 127 饱和和最大池化。FC2 保留有符号 INT32 分值，不重复套用中间层激活。

模型建议摘要：验证分为网络输入、逐层张量和最终分值三个层次。最终 argmax 相同只能说明该次类别相同，不能证明 22 个分值及中间计算一致；应定位最早出现差异的层，进一步检查累加、量化或池化。模型准确率由开发验证集评价，部署一致性则由相同输入下的整数输出对拍评价，两项结论分别报告。

实施与方案校正：团队将 QAT、导出工具、整数参考与 RTL 的舍入及饱和规则统一，保持 64×64 UINT8 输入的方向、灰度和行优先布局；整网以一次 START 完成计算，层间特征留在片内。第三层改为四输出通道并行时，仍保留每个输出通道原有累加顺序，复用公共量化器，避免因并行结构变化引入新的算术差异。

验证结果：部署验收核对逐层整数结果和 22 维分值。量化模型在 910 张自制开发验证图像中正确识别 826 张，准确率 90.77%；四路优化阶段 66 组第三层专项回归、4 次完整网络逐层对拍和 132 组板端输出检查均与参考一致。其中 132 组记录来自 44 张固定输入重复 3 轮，支持所测输入的部署一致性，不替代独立图像集的准确率评估。

5.5 统一 RTL 的 DDR 模型缓存与安全取消

团队提问：将 Conv3 和全连接纳入统一手写 RTL 后，如何在片内存储有限时分块读取 DDR 参数并复用缓存？遇到软件超时、用户取消或 AXI 响应错误，怎样避免留下未完成事务、提前释放模型缓冲，或把上一轮结果误作为当前有效输出？DONE、BUSY 和总线静止是否需要分别判定？

模型建议摘要：使用只读 AXI 主接口按块装入模型，片内保留当前权重块和偏置，并按生命周期安排 Pool2、Pool3、hidden 及 logits。START 锁存配置，层间特征留在片内；整组 22 维分值在 COMMIT 时统一发布，输入完整性、结果有效性和总线空闲分别检查。

模型建议摘要：取消后停止发起新工作，并排空已经接受的事务。ARVALID 置高后应保持地址及控制载荷，直到 ARREADY 握手；已经握手的 burst 继续按协议接收。正常排空的错误进入安全错误状态，RID 或 RLAST 等协议污染则保留故障与缓冲所有权。释放 DDR 前，应确认无待握手请求、未完成 burst、返回队列及计算流水，不能仅凭 BUSY 清零或重启 Notebook 判断安全。

实施与方案校正：统一 RTL 基础阶段采用 1KiB 共享模型缓存，权重区 512B，偏置区 512B 且有效偏置 472B；后续四路 Conv3 另增加局部权重缓存，见 5.6。实现中修正超时判断与 RREADY 的组合反馈、ABORT 同拍接受新读，以及取消后仍发布排队缓存响应的问题。控制面直接核对 pending/outstanding，ABORT 与 COMMIT 同拍时优先取消，防止旧结果重新有效。

验证结果：S2 读取器完成 71 组正常读取，并另行覆盖背压、RRESP、RID、提前或缺失 RLAST、超时及取消等检查。模型加载器完成 359 次逻辑装载，读取 169720B 有效模型数据并逐字节核对。后续整网继续验证可恢复错误、协议污染保留所有权，以及错误后完整 22 维分值恢复。

证据范围：S2 属于模块 RTL 仿真，后续恢复检查属于整网仿真；控制测试台提供的后端事件不作为真实 CNN 预测。协议污染场景之间的隔离复位也不表示运行时可以随意局部复位并释放缓冲，取消安全性以协议排空和资源所有权证明为依据。

5.6 Conv3 四输出通道并行与整网集成纠错

团队提问：统一 RTL 初版已能完成整网推理，但单路 Conv3 成为主要瓶颈，完整调用尚未优于同板 PS 优化 C。如何在 BRAM 紧张的 Zynq-7020 上提高第三层吞吐，同时保持模型、输入接口、DDR 参数布局及数值行为？并行结果的提交顺序变化后，原写地址保护如何调整，才能兼顾正确性与错误检测？

模型建议摘要：并行不同输出通道，将同一输入广播到四路 MAC，各路使用独立权重与累加器，并保持各通道的 IC/KY/KX 累加顺序。Pool2 只保留一份，以本地缓存重用当前通道组权重，继续共享量化器。先做第三层专项验证，再接入整网、完成布局布线，最后用同板 PS C 和 RTL 配对测量确认收益。

实测反馈与方案调整：初版先完成整网功能与资源收敛，首轮实测为 PS 优化 C 平均 49.394ms、单路 RTL 66.558ms。团队据此将优化集中到第三层实际运算与供数，新增一块 RAMB18 缓存四组权重，采用四路输出通道并行和输入广播。MAC 实际映射 LUT，因此“四路”不能写成新增四个 DSP；应以综合及完整实现报告确认代价。

整网集成与自我纠错：首个候选触发错误码 9：第三层按通道组交错提交结果，旧顶层仍按单路连续地址验证。团队保留地址保护，改为按通道组计算期望写地址，继续检查一一覆盖及越界；Pool3 最终 CHW 存储和 FC 读取格式保持不变，并新增错误写地址故障测试。恢复验收也从仅执行取消的冒烟检查，补足为错误后运行完整 Golden 推理并核对 22 维分值。

数值与实现验证：第三层累计 66 组回归，检查 1824768 个累加、激活和池化值；4 次完整网络逐层核对检查 1438168 个层输出，均与参考一致。完整整机 LUT 由 27563 增至 27921，FF 由 36329 增至 36726，BRAM 由 128 增至 128.5，DSP 保持 123；已约束路径 WNS 为＋0.119ns、WHS 为＋0.011ns。

同板性能验证：同一 Zynq-7020 上，22 类各选 2 张冻结输入，预热后交替执行三轮，取得 132 组配对记录，全部 22 维整数分值与参考一致。PS 优化 C 平均 48.933ms，四路 RTL 完整调用 26.680ms，实现 1.83 倍加速、耗时下降 45.48%。RTL 计时包含输入传输与校验、启动、等待、结果读取及 ACK，不含摄像、预处理、显示和初始化；历史单路 66.558ms 与四路 26.680ms 来自两次会话，其约 2.49 倍变化单列为优化趋势。

5.7 稳定帧交接约束下的预览调度优化

团队提问：CNN 推理加速后，摄像预览仍存在更新不够连贯的问题。如何区分采帧等待、内存复制、三维渲染和显示提交的耗时，避免将 HDMI 扫描频率等同于实际画面更新率？在不改变 Block Design、模型和正式识别链的前提下，能否通过等待重叠改善预览，同时保留完整帧、写者切换及缓冲区所有权检查？

模型建议摘要：先在摄像预览与固定三维负载共存的场景进行短时分段计时，定位主耗时，再修改对应调度。将物理显示扫描、预览提交频率和 CNN 调用耗时分别统计。若主要开销来自顺序等待，可分析新帧事件与 park 切换确认的依赖，重叠能够同时推进的等待；不能直接删去帧完成或写者离开的安全条件。

诊断与优化实施：轻量诊断显示采帧等待是主要耗时来源，团队据此将第二次新帧事件等待与 park 切换确认重叠，保留稳定帧与所有权判定。改动限定在 PS 预览调度，硬件、模型和识别计算链保持一致；验证也随影响范围收缩为针对性检查及串联功能确认，避免无关重建和重复测试。

证据复核与纠正：自动对照在保存阶段出现 NumPy 对象 JSON 序列化错误，随后断电，优化后完整 FPS 未能留存。团队修正报告序列化和运行入口，采用“已有数据说明到哪里，结论就写到哪里”的处理方式：保留实板观察到的流畅性改善，不以旧基线或主观感受计算帧率提升比例。

验证结果：现有记录支持预览流畅性改善及正常停止；不报告未经完整保存的优化后精确 FPS。该协作把性能诊断、调度安全与测试范围结合起来，使显示优化具有明确的实施边界，也避免把预览改进计为 CNN 推理加速收益。

5.8 异步结果一致性与硬件资源安全退出

团队提问：摄像识别、语音任务和网页命令的完成时刻不同。模式切换、取消或清空之后，怎样判定迟到结果是否仍属于当前请求，防止旧类别重新覆盖显示？停止时又应如何协调工作线程、DMA 及缓冲区生命周期，并设计可重复调用的停止入口，使后台任务退出与资源释放具有可检查的顺序？

模型建议摘要：为服务实例、模式版本和请求分配身份。结果发布前核对当前状态，过期结果丢弃；类别 0 必须与“无有效结果”分开表达。网页状态查询与耗时操作分离，避免录音或推理期间阻塞状态更新。共享硬件访问由统一应用调度及所有权约束管理，不能仅靠界面按钮状态判断任务是否结束。

模型建议摘要：退出先关闭新命令入口并取消或等待活动任务，停止预览、语音及相关硬件访问，确认 DMA 和缓冲区使用已结束后再释放内存；无法确认安全时保留资源与错误记录。停止结果应保存为本会话状态，重复执行可复用该记录，并与性能对照变量解耦。

实施与故障纠正：团队在请求及网页协议中使用 server_id、mode_epoch 和 request_id 等身份字段，结果发布前再次校验。联调中还发现旧停止单元在 session.stop()成功后因缺少 compare_result 继续报错；据此区分“应用停止失败”与“停止后报告处理失败”，修正停止入口及报告保存逻辑，避免把两类故障混为一谈。

验证结果：已有停止报告显示 errors 为空，13 个缓冲区全部释放，active_bos 与 active_bytes 均为 0，DMA 所有权释放，预览、语音及网页线程退出。摄像、语音、三维、网页及正常停止功能由人员实板确认；后续入口封装检查单独归档，不计为新增板测。该设计使请求有效性和资源回收能够通过状态与报告核验。

5.9 将校验经验提炼为可复用技能

团队提问：交付前需要反复核对 bit/HWH 及定点输出。怎样把这些检查提炼为其他工程可用的工具，同时保留版本来源和数值规则，避免脚本通过后被误认为整机已验证？

AI 协助要点：辅助拆分检查逻辑、工程配置和判定输出，梳理正常样例与单因素反例，提示区分文件一致、接口一致和板端功能三个层次。由团队确认配置来源、选取参考数据并复核结果。

采用的建议：将文件路径、哈希及 IP 地址移入 profile，保留独立的 Overlay/HWH 检查脚本；将整数输出比较独立为 NPY/NPZ 工具，严格核对形状、dtype 及元素值。采用正常样例和错误哈希、错误地址、单元素变化、dtype 变化反例，检查错误是否被准确检出。具体工具及证据见第六部分。

未采用或限制的方向：数组比较不加入静默类型转换、自动转置或浮点容差；技能包不承担自动加载硬件和故障恢复。文件检查通过仅证明与既定配置一致，未专项验证的分支、其他板卡和其他题目均不扩大为已验证能力。

团队心得：可复用工具需要同时保留输入约定、判定依据、错误出口和复验方法。尤其要先确认 profile 及 Golden 的来源，再讨论自动检查；两个文件相同只能证明一致性，参考本身是否正确仍需工程人员判断。

5.10 复现交付中的版本与运行状态管理

团队提问：怎样使队友拿到交付包后，能够依据同一套步骤启动摄像、语音及三维功能，并在停止后确认资源释放？遇到旧目录、Notebook 内核重启或启动中断，怎样判断可以继续的前提？

AI 协助要点：辅助梳理环境、上传、校验、解压、启动、功能操作和停止的依赖顺序，将真实上电状态与软件会话状态分别描述，并把报错处理改写为可执行的判断条件。团队负责核对实际包名、哈希、Notebook 入口及停止报告字段。

采用的建议：使用配套完整 ZIP 与解压脚本，先核对 SHA-256，再解压到新目录；根据真实断电或已正常停止的状态选择 cold_start 或 reuse，按顺序运行单元。停止结果与性能测试变量解耦，保存本会话报告。已有数据、预置资源和板端依赖随包核对，操作步骤集中写入第七部分。

未采用或暂缓的方向：未采用覆盖旧目录拼接不同版本文件、将内核重启视作硬件复位，或出错后自动反复加载 Overlay 的处理方式；对未确认静止的硬件不强制释放缓冲。本轮复现指南采用已构建运行包，未把重新训练或完整综合作为运行前置条件；源码重建属于另一层次的复现需求。

团队心得：复现说明要交代操作者的初始状态、每一步的成功标志和失败后的处理路径。一次能运行的演示需要进一步整理为可重复操作的交付；AI 适合梳理步骤，文件版本与硬件状态仍必须由真实记录核对。

5.11 性能指标与技术表达的人工复核

团队提问：如何把算法、硬件优化和系统效果写得清楚，同时保持指标的测量范围一致？哪些结果能够支撑加速、准确率和功能完成的结论，哪些表述需要收敛到当前已实现范围？

AI 协助要点：辅助整理性能对比表、摘要和展示脚本，按测量对象检查数值及说明是否对应，区分固定输入部署一致性、开发验证准确率、预览流畅性和整机交互效果。团队依据原始数据与功能实现逐项确认后采用。

采用的建议：以同板、同定点模型的 132 组配对记录陈述 1.83 倍 CNN 调用加速；保留 RTL 调用包含传输与控制、不含摄像预处理显示的说明。明确训练和验证数据均为自制，90.77%对应 910 张开发验证图像；将 22 类识别后调用预置模型、HDMI 与网页的实际交互能力作为功能亮点。

未采用或修正的表述：未将 1.83 倍外推为整机所有功能加速，未将 132 次重复输入记录写成 132 个独立样本；未把显示扫描频率当作预览帧率，也未填写缺少完整保存记录的优化后 FPS。对任意结构重建、自动生成未知分子模型，以及把 PS 语音 CNN 或三维渲染归为 PL 计算的表述，均按实际实现范围修正。

团队心得：技术表达的说服力来自可追溯的贡献与证据。将算法方法、RTL 架构和展示效果分别说明，再解释它们如何协同，比堆叠术语更便于理解；每一个显著数字都应能回答比较对象、输入条件、计时边界和原始记录在哪里。

5.12 团队对 AI 协作的复盘与心得

先把问题写成约束。我们逐步将需求整理为模型版本、张量布局、器件资源、接口协议和验收条件，再请 AI 分析候选方案。明确哪些内容必须保持不变，有助于得到能接入当前工程的建议，减少实现后才发现接口或数值规则不一致的返工。

以实测选择方案。初版统一 RTL 调用未优于 PS 优化 C，促使我们将问题收敛到 Conv3 的运算与供数；四路并行的收益最终由同板配对测试确认。由此形成的经验是先定位瓶颈，再讨论并行度，并同时检查访存、资源及布局布线后的时序代价。

用明确的判据组织验证。数值迁移核对整数张量与分值，协议改动检查握手与排空，调度改动检查帧和缓冲所有权，交付检查文件及启动状态。测试范围随改动影响确定，在保留必要证据的前提下减少无关重建和重复运行。

保留人的审核责任。AI 帮助展开思路、审查代码和整理材料；团队负责判断建议是否适合本工程，执行实板操作，核对证据并决定采用、修正或暂缓。我们更重视能够解释设计取舍、定位反例和复现结果的协作过程，而不以生成代码量或回答长度评价完成质量。

第六部分　技能包提炼过程

本项目围绕部署检查和数值验收两项重复需求，整理形成可复用校验技能包。包内仅保留 Overlay 文件与 HWH 接口检查、NPY/NPZ 数组严格比对两项工具，以及配套脚本、工程配置、说明和验证依据。

技能包入口为 42754/skill/README.md。

6.1 技能来源与通用逻辑提炼

第一项观察来自工程版本迭代：相近的文件名和相同的 IP 实例名，不能证明 bit、HWH 及软件预期地址属于同一份已确认配置。因此，将“文件是否为预期版本”和“接口地址是否一致”分为两类检查，分别使用 SHA-256 和 HWH 元数据判定。该结论属于工程观察，不将未经日志证实的情况写成实际发生的板端故障。

第二项观察来自定点 CNN 迁移与并行优化：最终类别相同，并不代表全部整数分值相同；形状、dtype 或有符号性变化也可能被自动类型转换掩盖。因此，将数值验收从 argmax 类别核对细化为键集合、形状、数据类型及逐元素精确相等检查。

提炼时将通用检查逻辑与本项目参数分离：文件读取、哈希计算、HWH 地址解析和数组比较写入独立脚本；bit/HWH 路径、期望哈希、IP 实例及地址放入 profiles/current.json；待比较数组由命令行传入。脚本不写死 22 类有机物、64×64 输入或 CNN 层名，也不导入 PYNQ、不访问 MMIO、不下载 bitstream。

6.2 Overlay 文件与 HWH 接口检查

validate_overlay.py 依赖 Python 3 标准库。工具检查文件存在性及目录范围，计算 bit/HWH 的 SHA-256，再解析 HWH 中 MEMRANGE 节点的 INSTANCE 和 BASEVALUE。配置内每个指定实例必须具有唯一的预期基地址，缺失、歧义或不符均返回失败。当前工程配置覆盖 6 个 IP；配置未列出的实例不属于本次检查范围。可在提交包根目录运行：

```
python -B skill/scripts/validate_overlay.py --profile
skill/profiles/current.json --root src/ps/psv7_pynq
```

验证采用正常样例与单因素反例：已确认的文件和 6 个 IP 地址能够通过；修改 bit 期望哈希后，工具报告文件哈希不符；仅将 CNN 预期地址偏移 4 字节后，工具报告地址缺失或歧义。正常样例预期退出码为 0，两类错误样例预期退

出码为 1。反例使用配置副本构造，不改动正式交付配置，也不作为历史上板事故。

6.3 NPY 与 NPZ 整数输出严格比对

compare_arrays.py 依赖 Python 3 和 NumPy，读取时禁用 pickle。NPY 作为一个完整数组比较；NPZ 先核对键集合，再逐键核对形状、dtype 和每个元素。工具不做广播、转置、布局重排、隐式类型转换或浮点容差处理，并输出 JSON 判定与差异数。使用示例如下，两个文件也可替换为 NPZ：

```
python -B skill/scripts/compare_arrays.py expected.npy actual.npy
```

验证以冻结 Golden 的 INT32 logits 构造三组样例：相同数组通过且差异数为 0；只改变一个元素时判定失败且差异数为 1；将 INT32 改为 INT64 时因 dtype 不符被拒绝。预期退出码依次为 0、1、1。该验证同时检查正常放行、错误检出和差异定位，避免仅用“脚本能运行”证明有效。

6.4 验证证据与复用边界

验证结果保存在 organization_verification.json（位于 sim/results/）中。记录标注 hardware_accessed=false、new_vivado_run=false，属于离线工具验证，不能计入板端 132 组配对测试。现有记录明确支持正常配置、错误哈希、错误地址、相同数组、单元素差异及 dtype 差异；未专项记录的代码分支不写成已经完成验证。

Overlay 工具只能证明文件与给定配置一致，不能证明配置本身正确，也不能确认板上当前加载内容、时钟、引脚、时序或 DMA 状态。数组工具只能判断两份数据是否一致，不能证明参考正确、数据来源真实或模型泛化能力；需要容差的算法应另行定义规则。

迁移到其他题目时，保留脚本并替换经确认的 profile 或参考数组，先复验正常样例和单因素反例，再进入该工程的实际加载与功能验收。更换 HWH 格式、文件格式或数值规则时，需要相应适配并补充验证；可配置接口只说明具备复用条件，不等于已在其他板卡完成上板验证。

第七部分　复现说明

按下文完成“打开工程 → 仿真 → 生成硬件 → 制作 ZIP → 上板验证”。复现从提供的最终模型参数开始。直接使用已有运行包时，从步骤 4 开始。

电脑端硬件重建需要 Vivado 2020.2（含 Zynq-7000 器件支持）、Python 3.8 或以上及 NumPy。标准构建直接使用随附的 PREP 导出 IP，因此本流程无需再次执行 Vitis HLS。

7.1 打开工程并运行仿真

本步在电脑上检查硬件的计算和接口。

1. 将完整提交包解压到短英文路径，在 Vivado **2020.2** 打开 build/project/camera_bridge.xpr。
2. 打开 system.bd 可查看整机连接。
3. 在 **Simulation Sources** 中右键目标仿真集，选择 **Make Active**。
4. 点击 **Run Behavioral Simulation → Run All**，等待以下通过标志，再点击 **Zoom Fit** 查看波形。切换测试前先关闭当前仿真。

• sim_network：测试完整 CNN，含 Conv1、Conv2、四路 Conv3 和全连接。通过标志为 S5_PASS_NETWORK。

• sim_prep：测试预处理接口及红色图案首行输出。通过标志为 S6_PREP_IDENTITY_RTL_PASS 和 PREP_PIXEL_PASS。

• sim_audio：测试 I²S 接收和 FIR 滤波抽取。通过标志为 AUDIO_PASS。

工程初始只运行 **1 µs**。继续执行 run all，完整 CNN 样例约在 **24.2423 ms 仿真时刻**结束，电脑实际需等待数分钟。.wcfg 会自动加载，日志和已有波形在 sim/results/gui/。

7.2 生成 bit/HWH

本步生成配套的 bit/HWH。**方式一和方式二任选一种，完成后都执行步骤 3 的同一个打包命令。**

方式一：打开提供的工程，在 Vivado 界面生成硬件

1. 在已打开的工程中点击 **Generate Bitstream**，等待生成成功。

2. 在**提交根目录**打开 Windows PowerShell，执行下列命令，把同一轮生成的 bit/HWH 复制成打包工具需要的名称。输出目录须尚不存在。

```
$out = Join-Path (Split-Path (Get-Location).Path -Parent)
'chem_gui_build_01'
if (Test-Path -LiteralPath $out) { throw 'Choose a new output directory' }
$bit = 'build/project/camera_bridge.runs/impl_1/system_wrapper.bit'
$hwh =
'build/project/camera_bridge.gen/sources_1/bd/system/hw_handoff/system.hwh'
Get-Item -LiteralPath $bit, $hwh
$hardwareOutput = Join-Path $out 'hardware'
New-Item -ItemType Directory -Path $hardwareOutput | Out-Null
Copy-Item -LiteralPath $bit -Destination
"$hardwareOutput/psv7_camera_audio.bit"
Copy-Item -LiteralPath $hwh -Destination
"$hardwareOutput/psv7_camera_audio.hwh"
```

完成方式一后，跳过方式二，在同一个 PowerShell 窗口执行步骤 3。

方式二：运行脚本，从源码自动构建硬件

选择方式二时，在提交根目录打开 PowerShell，修改 Vivado 安装路径。脚本调用 Vivado 创建工程、综合、布局布线并生成 bit/HWH。完成后继续执行步骤 3。

```
$vivado = 'C:/Xilinx/Vivado/2020.2/bin/vivado.bat'
$out = Join-Path (Split-Path (Get-Location).Path -Parent)
'chem_script_build_01'
python -B build/build_hardware.py --vivado "$vivado" --output "$out"
if ($LASTEXITCODE -ne 0) { throw 'Hardware build failed' }
$hardwareOutput = Join-Path $out 'run/output'
```

7.3 制作上板运行 ZIP

两种方式到这里汇合。本步使用同一个 prepare_runtime.py，把刚生成的 bit/HWH、PS 应用、模型和网页资源合成运行 ZIP。使用能导入 NumPy 的 Python，在步骤 2 的同一个 PowerShell 窗口执行：

```
python -B sim/check_parameters.py
if ($LASTEXITCODE -ne 0) { throw 'Parameter check failed' }
$runtimeZip = Join-Path $out 'chem_runtime.zip'
python -B board/prepare_runtime.py --hardware-output "$hardwareOutput" --
output "$runtimeZip"
if ($LASTEXITCODE -ne 0) { throw 'Runtime packaging failed' }
```

确认 passed 和 rebuilt_pair_used 均为 true。记下 ZIP 位置 $runtimeZip 和打印的 **SHA-256**。

7.4 上传、校验并解压

本步把运行包部署到真板。准备匹配的 Zynq-7020 板（xc7z020clg400-2）、OV5640、HDMI 显示器及 WM8960 板载麦克风，断电接线后上电。板端使用适配本板的 **PYNQ 2.7.0 / armv7l** 系统，并具备 Python、NumPy、GCC 和 Jupyter。提交包提供应用，不含 SD 卡系统镜像。

1. 让电脑和开发板接入可互通的局域网，打开 Jupyter。
2. 上传步骤 3 的 chem_runtime.zip 和 board/unpack_runtime.py 到 /home/xilinx/jupyter_notebooks/。
3. 打开 Jupyter **Terminal**，执行下列命令。把校验值替换为步骤 3 打印的完整值；输出目录已存在时换成新的目录名。

```
cd /home/xilinx/jupyter_notebooks
runtime_zip='chem_runtime.zip'
expected_sha='填入步骤 3 打印的 64 位 sha256'
ls -lh "$runtime_zip" unpack_runtime.py
sha256sum "$runtime_zip"
python3 unpack_runtime.py "$runtime_zip" --output chem_run_01 --sha256
"$expected_sha"
hostname -I
```

确认哈希一致，解压显示 host_assets_checked: 7 和 passed: true，记下开发板 IP。

直接使用已有运行包时

上传 board/deliveries/psv7_runtime.zip 和 board/unpack_runtime.py 后，在 Jupyter Terminal 执行：

```
cd /home/xilinx/jupyter_notebooks
runtime_zip='psv7_runtime.zip'
expected_sha='3afa8487ca95f5ffebbac951c3009b630e038215533ef57625fa5f5ce67141
68'
sha256sum "$runtime_zip"
python3 unpack_runtime.py "$runtime_zip" --output chem_run_01 --sha256
"$expected_sha"
hostname -I
```

确认校验和解压通过后，继续步骤 5。

7.5 启动作品

本步加载硬件并启动应用。

1. 在 Jupyter 中打开本次解压目录下的 psv7_pynq/notebooks/01_application.ipynb。如有旧应用，先在其 Notebook 正常停止并关闭旧内核。

2. 逐个执行代码单元。第一个单元完成文件检查。

3. 在第二个单元填写实际 IP，并根据开发板状态设置启动参数。

真实断电上电后首次启动：

```
STARTUP_MODE = 'cold_start'
CLEAN_BOARD_POWER_UP = True
BOARD_IP = '192.168.2.99'  # 改为实际 IP
```

旧应用已正常停止、此后未断电：

```
STARTUP_MODE = 'reuse'
CLEAN_BOARD_POWER_UP = False
BOARD_IP = '192.168.2.99'  # 改为实际 IP
```

执行第二个单元，等待无异常结束并返回 host_running: True、端口 8765。第三个单元留到使用结束时执行。

7.6 检查摄像、语音和三维展示

本步确认真实输入能驱动分子展示。

1. 在电脑浏览器打开 http://开发板实际 IP:8765，点击“连接开发板”。

2. 手动选择一种分子，核对名称、化学式和网页/HDMI 模型；用鼠标旋转、缩放。

3. 切到摄像模式，将支持类别的结构简式放入摄像头框内，点击“开始识别”，核对结果。

4. 切到语音模式，选择“板载麦克风”，点击“开始识别”并说出支持的有机物名称，核对结果。

5. 保存实际功能截图。

7.7 运行板端固定输入验证

本步检查 PS 与 PL 完整 CNN 输出是否一致。保持应用运行，在**同一个 Notebook 内核**新增单元，执行下面代码。测试期间暂停网页识别、录音和模式切换。

```
import sys
sys.path.insert(0, str(PROJECT_ROOT.parent / 'ps_vs_rtl'))
from board_benchmark import run
benchmark_result = run(session.app)
print(benchmark_result['passed'])
print(benchmark_result['report_directory'])
```

通过标准为 passed=True，44 份输入重复 3 轮，共 132 条记录的 22 个分值逐项一致。结果文件为打印目录中的 board_result.json，计时范围是 CNN 完整调用。

7.8 正常停止并下载结果

本步关闭任务和硬件，保存复现记录。

1. 执行原 Notebook 的第三个代码单元。

2. 确认错误列表为空、活动缓冲区为 0、分配与释放数量相等、DMA 和后台线程已停止。

3. 使用单元提供的链接下载停止报告。再从步骤 7 的结果目录下载 board_result.json，一并保留功能截图和实际运行 ZIP 的 SHA-256。

4. 摄像、语音、网页/HDMI、固定输入对拍和正常停止均通过后，本次整机复现完成。

停止失败时保留原内核，依据停止报告和 skill/README.md 排查。